Group 5

Practical Software Testing and Quality Assurance

Restaurant Table Reservation System (RTS)

Final Project Report

สมาชิก

6733062221	ณัฏฐณิชา จามจุรี	QA Lead / BA
6733270421	สิตานัน เยาวยัง	Test Designer
6633209121	รัชชานนท์ มุขแก้ว	Test Designer
6733272721	สิปปวิชญ์ แช่มชมดาว	Dev in Test
6733171021	พัชรมน สีห์รา	API Tester
6733064521	ณัฏฐามาศ อุดมสันติธรรม	Performance Tester
6733066821	ณัฐดนัย จรัสวิบูลย์กุล	Security Tester

Content

Content 3
1. Introduction 6
2. Scope of System 6
3. User Roles 6
3.1 Customer 6
3.2 Staff 6
3.3 Admin 7
4. Functional Requirements 7
4.1 Authentication 7
FR-01: User Registration 7
FR-02: User Login 7
4.2 Reservation Management 7
FR-03: View Available Tables 7
FR-04: Create Reservation 7
FR-05: Reservation Confirmation 8
FR-06: Modify Reservation 8
FR-07: Cancel Reservation 8
FR-08: Notification System 8
4.3 Table Management 8
FR-09: Table Status Management 8
FR-10: Double Booking Prevention 8
4.4 Walk-in Management 8
FR-11: Walk-in Reservation 8
4.5 Restaurant Configuration 8
FR-12: Restaurant Profile Management 8
5. API Specification (Custom Mock Design) 9
5.1 Base Information 9
5.2 Authentication 9
Register 9
Login 10
5.3 Reservation 10
View Availability 10
Make Reservation 11
Modify Reservation 12
Cancel Reservation 12
Walk-in Reservation 13
View History 13
5.4 Admin 14
View All Reservations 14
View Notifications 15

Update Table Status 16
Update Restaurant Info 16
Get All Tables 17
Create Table 18
Update Table 18
Delete Table 19
5.5 User 19
Get User Profile 19
5.6 Restaurant 20
Get Restaurant Info 20
6. Test Case Design 20
6.1 Use Case Diagram 21
6.2 Test Case Design Scope 21
6.3 Test Cases 21
7. API Testing 22
7.1 Scope 22
7.2 Approach 22
7.3 API Test Script 22
7.3.1 Test Case Classification 22
Admin (System Admin) 22
Authentication 22
Admin Control 22
Table Management 23
Reservation Management 23
Staff 23
Authentication 23
Reservation Management (Walk-in) 23
Table Management 23
User (Customer) 24
Authentication 24
Reservation 24
Table (Availability) 24
Overview 24
7.3.2 Testing Samples 25
Authentication APIs - Login 25
Reservation APIs - Create Reservation 25
Table Management (Staff) - Get Table List 26
Admin APIs - View All Reservations 26
Error Responses 27
7.3.3 Test Summary 27
Overall Test Results 27

Happy Path Results 27
Negative Path Results 27
System Limitations (Mock Data) 28
Conclusion 28
8. Automated Test 28
8.1 Framework Design 28
8.2 Test Data 28
8.3 Test Scenarios 29
8.3 Test Script 29
8.4 Test Result 29
8.5 Summary 30
9. Performance Testing 30
9.1 Test Scenario 30
9.2 Performance Criteria 30
9.3 Tool 30
9.4 Performance Test Script 31
9.5 Performance Testing Results Summary 31
9.6 Requirements vs. Actual Results 31
9.6.1 Concurrent Users: The system must support 1,000 concurrent users. 31
9.6.2 Error Rate: The error rate must be less than 1%. 36
9.6.3 Duration: The system must run smoothly for 1 hour (Soak Test). 37
9.6.4 Response Time Requirement 38
9.7 Summary 40
10. Security Testing 41
10.1 Scope 41
10.2 Security Test Approach 41
10.3 Security Test Result 41
10.3.1 Password Policy 41
10.3.2 Login Attempt Limitation 43
10.3.3 Token Expiration 46
10.3.4 Input Validation Security 47
10.3.4.1 SQL Injection Testing 47
10.3.4.2 Cross-Site Scripting (XSS) Testing 49
10.3.4.3 Business Logic 50
10.3.5 API Security 53
10.3.5.1 Vertical Privilege Escalation 53
10.3.5.2 Horizontal Privilege Escalation 55
10.3.5.3 Mass Assignment Testing 56
10.3.6 Sensitive Data Protection 58
11. Appendix / Reference 59
12. Conclusion 59

1. Introduction

The Restaurant Table Reservation System (RTS) is a web-based application designed to simulate a real restaurant booking system where customers can reserve tables, staff can manage reservations and table status, and administrators can configure restaurant information.

This project is implemented as a mock system created for software testing practice. The system is intentionally not fully stable and may contain multiple defects and bugs, which are used as part of testing activities such as functional testing, API testing, automation testing, performance testing, and security testing.

The purpose of this system is to provide a realistic environment for practicing requirement analysis, test design, and defect identification in a controlled project scenario.

2. Scope of System

The system supports:

- Customer reservation management (create / update / cancel)
- Staff reservation handling and table status updates
- Admin configuration and reporting
- Real-time table availability updates

3. User Roles

3.1 Customer

- Register / Login
- Create reservation
- Modify reservation
- Cancel reservation
- View reservation history

3.2 Staff

- View all reservations
- Create walk-in reservations
- Update table status (Available / Occupied / Out of Service)
- Receive reservation notifications

3.3 Admin

- Manage restaurant profile (name, hours, contact)
- Manage tables
- View system reports and statistics

4. Functional Requirements

4.1 Authentication

FR-01: User Registration

- System shall allow customer registration with:
    - Full Name (2–60 characters, letters only)
    - Email (valid format, unique)
    - Password (8–20 characters, must include uppercase + number)
    - Phone Number (10 digits)

FR-02: User Login

- System shall authenticate user using email and password
- Invalid credentials shall return error message
- System shall generate JWT token on success

4.2 Reservation Management

FR-03: View Available Tables

- System shall show available tables based on:
    - Date (future only)
    - Time (within operating hours)
    - Guest count (1–10)

FR-04: Create Reservation

- Customer can create reservation with:
    - Date (YYYY-MM-DD, future only)
    - Time (HH:mm, restaurant open hours)
    - Guest count (1–10 persons)
    - Special request (optional, max 200 chars)
- System shall assign available table automatically

- System shall prevent double booking

FR-05: Reservation Confirmation

- System shall return reservation ID and status = CONFIRMED

FR-06: Modify Reservation

- Customer can update reservation before reservation time
- System shall re-check table availability

FR-07: Cancel Reservation

- Customer can cancel at least 1 hour before reservation time
- System shall release table automatically

FR-08: Notification System

- Staff shall receive notifications when reservation is created, updated, or cancelled

4.3 Table Management

FR-09: Table Status Management

- Staff can update table status:
    - AVAILABLE
    - OCCUPIED
    - OUT_OF_SERVICE

FR-10: Double Booking Prevention

- System shall ensure no overlapping reservations for same table/time

4.4 Walk-in Management

FR-11: Walk-in Reservation

- Staff can create reservation on behalf of customer

4.5 Restaurant Configuration

FR-12: Restaurant Profile Management

- Admin can update:
    - Restaurant name
    - Operating hours
    - Contact information

5. API Specification (Custom Mock Design)

5.1 Base Information

- Base URL: `https://localhost:3000/api`
- Format: JSON

5.2 Authentication

Register

Endpoint: /auth/register

Method : POST

Access by : Guest User

Description : Register a new customer account into the system.

Note :

- Email must be unique
- Password must meet security rules
- Full name allows only letters and spaces (2–60 chars)
- Phone number is optional (10–15 digits)

Request Example :

```
{
  "fullName": "John Smith",
  "email": "john@email.com",
  "password": "Password123",
  "phoneNumber": "0812345678"
}
```

Response 201 :

```
{
  "userId": 101,
  "message": "Registration successful"
}
```

Response 400 :

```
{
  "error": "Invalid email format"
}
```

Response 409 :

```
{
  "message": "Email already exists"
}
```

Login

Endpoint : /auth/login

Method : POST

Access by : Guest User

Description : Authenticate user and return access token.

Note :

- Email and password are required
- Returns mock JWT token
- Invalid credentials return 401

Request Example :

```
{
  "email": "john@email.com",
  "password": "Password123"
}
```

Response 200 :

```
{
  "token": "mock-jwt-token",
  "role": "customer"
}
```

Response 400 :

```
{
  "error": "Email and password are required"
}
```

Response 401:

```
{
  "error": "Invalid credentials"
}
```

5.3 Reservation

View Availability

Endpoint : /reservations/availability

Method : GET

Access by : Guest User

Description : Check available tables based on date, time, and guest count.

Note :

- Must provide `date`, `time`, `guestCount`
- Returns only non-reserved tables

Request Example :

```
/reservations/availability?date=2026-04-17&time=18:00&guestCount=2
```

Response 200:

```
{
  "date": "2026-04-17",
  "time": "18:00",
  "guestCount": 2,
  "availableTables": [
    {
      "id": 1,
      "capacity": 4
    }
  ]
}
```

Response 400 :

```
{
  "error": "Missing required fields"
}
```

Make Reservation

Endpoint : /reservations

Method : POST

Access by : Customer

Description : Create a new reservation for an available table.

Note :

- Only customer role allowed
- Table must be available at selected time

Request Example :

```
{
  "date": "2026-04-17",
  "time": "18:00",
  "guestCount": 2,
  "specialRequest": "Near window"
}
```

Response 201:

```
{
  "id": 1713330000000,
  "userId": 5,
  "customerName": "John Smith",
  "customerEmail": "john@email.com",
  "tableId": 2,
  "date": "2026-04-17",
  "time": "18:00",
  "guestCount": 2,
  "specialRequest": "Near window",
  "status": "CONFIRMED",
  "createdByRole": "customer"
```

```
}
```

Response 400 :

```
{
  "error": "Missing required fields"
}
```

Response 409:

```
{
  "error": "No table available"
}
```

Modify Reservation

Endpoint : /reservations/**:id**

Method : PUT

Access by : Customer (Owner) / Admin

Description : Update reservation details.

Note :

- Only owner or admin can modify
- Partial update supported

Request Example :

```
{
  "date": "2026-04-18",
  "time": "19:00"
}
```

Response 200:

```
{
  "message": "Updated",
  "reservation": {
    "id": 1713330000000,
    "date": "2026-04-18",
    "time": "19:00"
  }
}
```

Response 403 :

```
{
  "error": "Forbidden"
}
```

Cancel Reservation

Endpoint : /reservations/**:id**

Method : DELETE

Access by : Customer (Owner) / Staff / Admin

Description : Cancel an existing reservation.

Note :

- Reservation will be removed permanently

Response 200:

```
{
  "message": "Deleted"
}
```

Response 404:

```
{
  "error": "Not found"
}
```

Walk-in Reservation

Endpoint : /reservations/walk-in

Method : POST

Access by : Staff / Admin

Description : Create reservation for walk-in customers.

Note :

- Only staff/admin allowed
- No userId required

Request Example:

```
{
  "customerName": "Walk-in Guest",
  "date": "2026-04-17",
  "time": "18:00",
  "guestCount": 3
}
```

Response 201:

```
{
  "id": 1713330000000,
  "userId": null,
  "customerName": "Walk-in Guest",
  "tableId": 3,
  "date": "2026-04-17",
  "time": "18:00",
  "guestCount": 3,
  "status": "CONFIRMED",
  "createdByRole": "staff"
}
```

Response 409:

```
{
  "error": "No table available"
}
```

View History

Endpoint : /reservations/history

Method : GET

Access by : Customer (Owner)

Description : Retrieve reservation history for the logged-in customer only.

Note :

- Only customer can access this endpoint
- Staff and Admin must use /admin/* endpoints instead
- Returns only the logged-in user's reservations

Response 200:

```
{
  "count": 2,
  "reservations": [
    {
      "id": 1713330000000,
      "userId": 5,
      "customerName": "John Smith",
      "tableId": 2,
      "date": "2026-04-17",
      "time": "18:00",
      "guestCount": 2,
      "status": "CONFIRMED"
    }
  ]
}
```

Response 403:

```
{
  "error": "Access denied. Use admin API instead."
}
```

5.4 Admin

View All Reservations

Endpoint : /admin/reservations

Method : GET

Access by : Staff / Admin

Description : Retrieve all reservations for management purposes.

Note :

- Admin and Staff can access
- Used for operational overview
- Customer data access is not allowed here

Response 200:

```
{
  "count": 2,
  "reservations": [
    {
      "id": 1713330000000,
      "tableId": 2,
      "date": "2026-04-17",
      "time": "18:00",
      "guestCount": 2,
      "status": "CONFIRMED",
      "createdByRole": "customer",
      "customer": {
        "userId": 5,
        "fullName": "John Smith",
        "email": "john@email.com",
        "phoneNumber": "0812345678"
      }
    }
  ]
}
```

Response 403:

```
{
  "error": "Forbidden"
}
```

View Notifications

Endpoint : /admin/notifications

Method : GET

Access by : Staff / Admin

Description : Get all system notifications related to reservations and system events.

Response 200:

```
{
  "count": 2,
  "notifications": [
    {
      "id": 1,
      "type": "RESERVATION_CREATED",
      "message": "New reservation created for Table #3",
      "reservationId": 12,
      "tableId": 3,
      "createdBy": {
        "userId": 101,
        "role": "customer",
        "name": "John Doe"
      },
      "createdAt": "2026-04-17T10:00:00Z",
      "read": false
    },
    {
      "id": 2,
```

```
      "type": "RESERVATION_UPDATED",
      "message": "Reservation #10 was updated to CONFIRMED",
      "reservationId": 10,
      "tableId": 5,
      "createdBy": {
        "userId": 201,
        "role": "staff",
        "name": "Staff A"
      },
      "createdAt": "2026-04-17T10:10:00Z",
      "read": true
    }
  ]
}
```

Update Table Status

Endpoint : /admin/tables/:id/status

Method : PUT

Access by : Staff / Admin

Description : Update table availability status

Note :

- Only valid status allowed:
  - AVAILABLE
  - OCCUPIED
  - OUT_OF_SERVICE

Request Example:

```
{
  "status": "OCCUPIED"
}
```

Response 200:

```
{
  "message": "Table status updated",
  "table": {
    "id": 1,
    "status": "OCCUPIED"
  }
}
```

Response 400:

```
{
  "error": "Invalid status"
}
```

Response 404:

```
{
  "error": "Table not found"
}
```

Update Restaurant Info

Endpoint : /admin/restaurant

Method : PUT

Access by : Admin

Description : Update restaurant configuration including name, operating hours, and contact information.

Note :

- Admin only
- Partial update supported
- Validates operating hours and contact format

Request Example:

```
{
  "name": "New Restaurant Name",
  "openingHour": "09:00",
  "closingHour": "21:00",
  "contact": {
    "phone": "0812345678",
    "email": "new@email.com"
  }
}
```

Response 200:

```
{
  "message": "Restaurant info updated",
  "restaurant": {}
}
```

Response 400:

```
{
  "error": "Invalid operating hours"
}
```

Response 403:

```
{
  "error": "Forbidden"
}
```

Get All Tables

Endpoint : /admin/tables

Method : GET

Access by : Staff / Admin

Description : Get all tables in restaurant system for management

Note :

- Requires authentication token

Response 200:

```
{
  "count": 3,
  "tables": [
    {
      "id": 1,
      "capacity": 4,
      "status": "AVAILABLE"
    }
  ]
}
```

Response 403:

```
{
  "error": "Forbidden"
}
```

Create Table

Endpoint : /admin/tables

Method : POST

Access by : Admin

Description : Create a new table in restaurant system

Note :

- capacity must be greater than 0

Request Example :

```
{
  "capacity": 4,
  "status": "AVAILABLE"
}
```

Response 201:

```
{
  "message": "Table created",
  "table": {
    "id": 4,
    "capacity": 4,
    "status": "AVAILABLE"
  }
}
```

Response 400:

```
{
  "error": "Invalid capacity"
}
```

Update Table

Endpoint : /admin/tables/:id

Method : PUT

Access by : Admin

Description : Update table information (capacity or status)

Note :

- Partial update supported

Request Example :

```
{
  "capacity": 6,
  "status": "OCCUPIED"
}
```

Response 200 :

```
{
  "message": "Table updated",
  "table": {
    "id": 1,
    "capacity": 6,
    "status": "OCCUPIED"
  }
}
```

Response 404:

```
{
  "error": "Table not found"
}
```

Delete Table

Endpoint : /admin/tables/:id

Method : DELETE

Access by : Admin

Description : Remove table from system

Note :

- Cannot delete table if it is currently reserved (optional rule)

Response 200 :

```
{
  "message": "Table deleted"
}
```

Response 404:

```
{
  "error": "Table not found"
}
```

5.5 User

Get User Profile

Endpoint : /users/profile

Method : GET

Access by : Customer / Staff / Admin

Description : Retrieve user profile information from the system.

Note :

- Currently returns mock user (first user in database)
- In real system, should return authenticated user via token
- Used for displaying basic user info

Response 200:

```
{
  "id": 1,
  "email": "john@email.com",
  "role": "customer"
}
```

Response 404:

```
{
  "error": "User not found"
}
```

5.6 Restaurant

Get Restaurant Info

Endpoint : /restaurant

Method : GET

Access by : Public

Description : Retrieve basic restaurant information such as name, opening hours, closing hours, and contact details.

Note :

- Public endpoint (no authentication required)
- Used for displaying restaurant info on homepage or app landing page

Response 200:

```
{
  "name": "Restaurant Name",
  "openingHour": "10:00",
  "closingHour": "22:00",
  "contact": {
    "phone": "0812345678",
    "email": "restaurant@email.com"
```

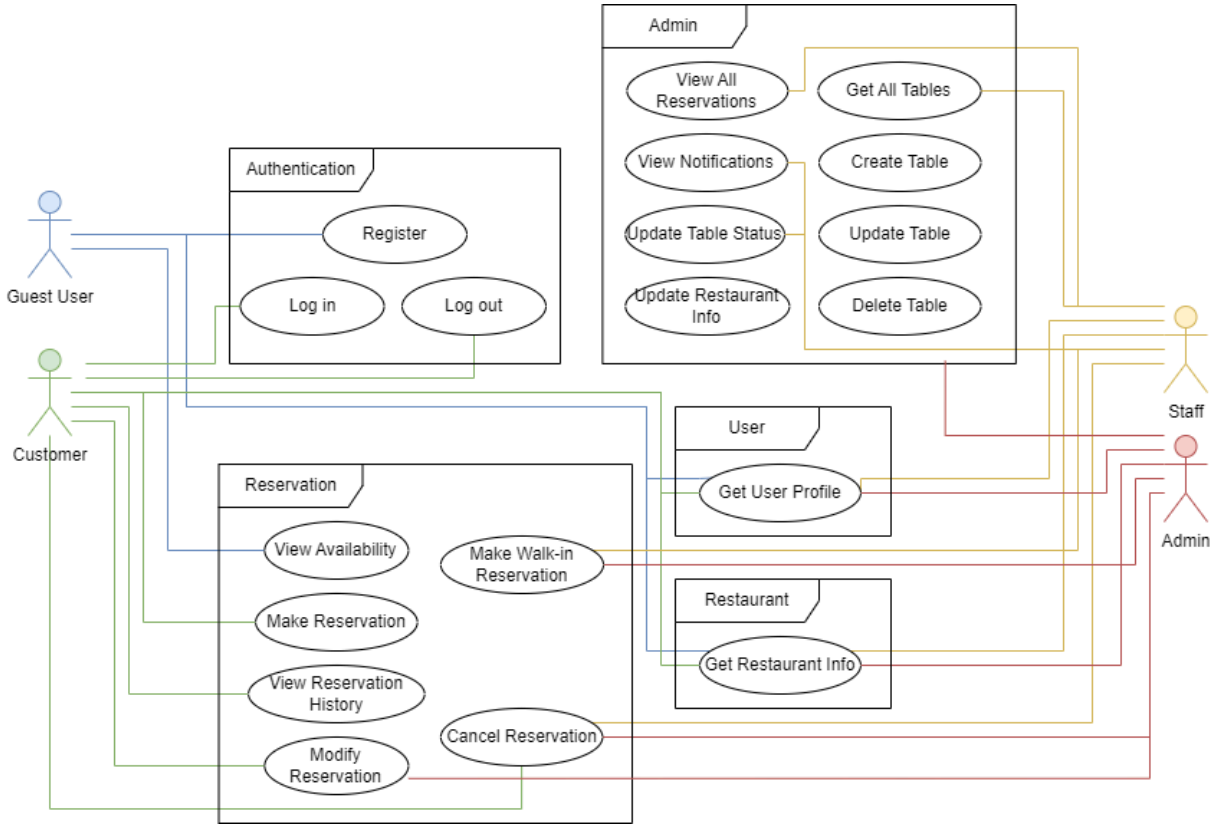
```
  }
}
```

6. Test Case Design

This section defines test cases derived from system requirements and user workflows (Customer / Staff / Admin). Software testing techniques applied:

- Equivalence Partitioning (EP)
- Boundary Value Analysis (BVA)
- Decision Table Testing
- Negative Testing

6.1 Use Case Diagram



6.2 Test Case Design Scope

- Functional Test Cases
- Boundary Test Cases
- Negative Scenarios
- Role-based Access Test Cases

6.3 Test Cases

Test Cases design sheet : Test_Case_Design

7. API Testing

API testing is performed based on the REST API specification to validate backend functionality.

7.1 Scope

- Authentication APIs (/auth/register, /auth/login)
- Reservation APIs (/reservations)
- Table APIs (/tables)
- Admin APIs (/admin/*)

7.2 Approach

- Positive & Negative API testing
- Status code validation
- Response schema validation
- Authentication & authorization testing

7.3 API Test Script

API Test Script

7.3.1 Test Case Classification

This collection is divided into three main testing groups based on user roles:

Admin (System Admin)

Authentication

- **Happy Path**
  - Login Admin
- **Negative Path**
  - Login - Invalid Credentials
  - Login - Non-admin Account

Admin Control

- **Happy Path**
  - Update Restaurant Info
  - Get Notifications

- **Negative Path**
    - Update Restaurant - Invalid Name
    - Update Restaurant - Invalid Phone
    - Update Restaurant - Invalid Email
    - Update Restaurant - Invalid Hours
    - Update Restaurant - Unauthorized (Not Admin)
    - Get Notifications - Forbidden

Table Management

- **Happy Path**
    - Get All Table
    - Create Table
    - Update Table
    - Delete Table
- **Negative Path**
    - Create Table - Required Capacity
    - Create Table - Invalid Capacity
    - Update Table - Table Not Found
    - Delete Table - Table Not Found

Reservation Management

- **Happy Path**
    - Get All Reservation
    - Update Reservation
    - Cancel Reservation
- **Negative Path**
    - Update/Cancel - Reservation Not Found

Staff

Authentication

- **Happy Path**
    - Login Staff
- **Negative Path**
    - Login - Invalid Credentials

Reservation Management (Walk-in)

- **Happy Path**
    - Create Reservation (Walk-in)
    - Get All Reservation
    - Update Reservation (Walk-in)
    - Cancel Reservation (Walk-in)
- **Negative Path**
    - Create Reservation - Unauthorized Staff
    - Update/Cancel - Reservation Not Found

Table Management

- **Happy Path**
  - Get All Table
  - Update Table Status
- **Negative Path**
  - Update Table - Invalid Status
  - Update Table - Table Not Found

User (Customer)

Authentication

- **Happy Path**
  - Register Customer
  - Login Customer
- **Negative Path**
  - Register - Duplicate Email
  - Register - Invalid Email Format
  - Register - Invalid Full Name
  - Register - Invalid Phone Number
  - Login - Wrong Email
  - Login - Wrong Password

Reservation

- **Happy Path**
  - Create My Reservation
  - Get My Reservation
  - Update My Reservation
  - Cancel My Reservation
- **Negative Path**
  - Create - Double Booking (Overlap)
  - Create - Missing Required Fields
  - Create - Outside Operating Hours
  - Create - Over Capacity
  - Create - Past Date
  - Get - Deleted Reservation
  - Update - After The Reservation Time
  - Update - Deleted Reservation
  - Cancel - Less than 1hr before time
  - Cancel - Deleted Reservation

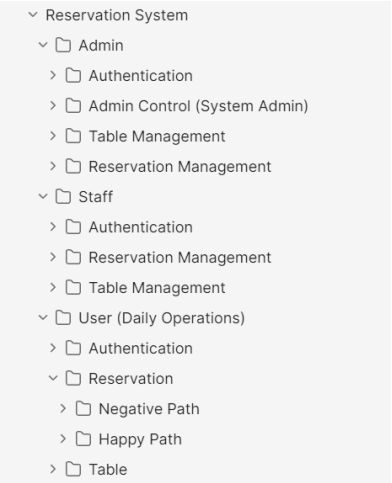
Table (Availability)

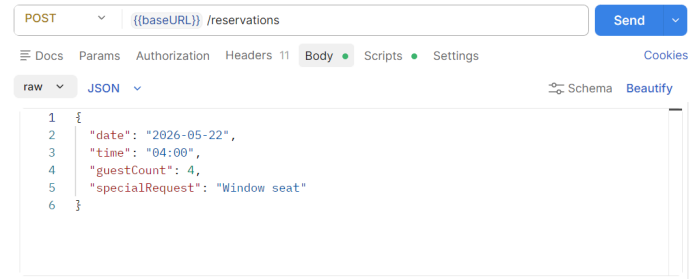
- **Happy Path**
  - Get Available Table
- **Negative Path**
  - Get - Missing Required Fields
  - Get - Invalid Guest Count

Overview

This collection provides comprehensive testing coverage for both **Happy Path** and **Negative Path** scenarios across all system roles, including **Admin**, **Staff**, and **User**. The test suite focuses on the following core functionalities:

- **Authentication:** Registration and Login security across different roles.
- **Reservation Management:** Booking lifecycle from creation to cancellation.
- **Table Management:** Availability tracking and physical table configuration.
- **Restaurant Settings:** System-wide configurations and administrative controls.

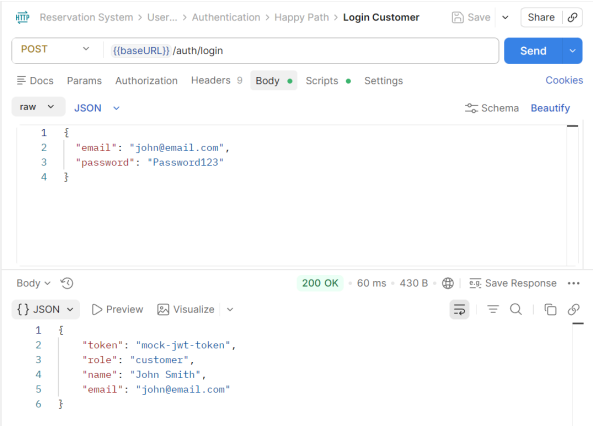




7.3.2 Testing Samples

Here are 5 Example Test Situations based on requirements

Authentication APIs - Login



Reservation APIs - Create Reservation

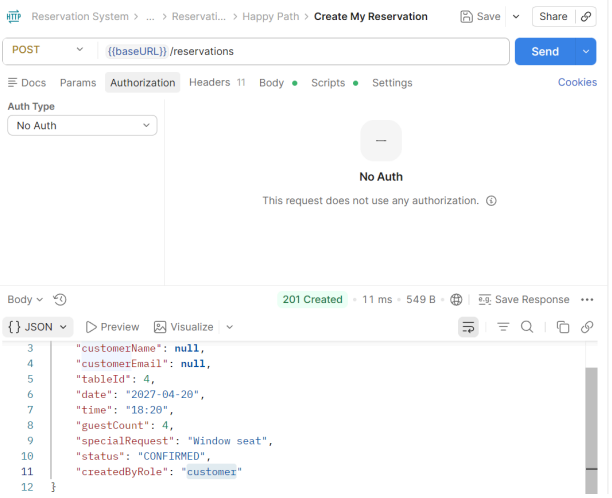


Table Management (Staff) - Get Table List

Reservation System > User (Dail... > Table > Happy Path > **Get Available Table**

GET {{baseURL}} /reservations/availability?date=2026-04-20&time=18:00&guestCo...

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Pre-req
Post-res

```
const res = pm.response.json();
const query = pm.request.url.query.toObject();


pm.response.to.have.status(200);


pm.expect(res.date).to.not.be.undefined;
pm.expect(res.time).to.not.be.undefined;
pm.expect(res.guestCount).to.not.be.undefined;
pm.expect(res.availableTables).to.not.be.undefined;

```

Body 200 OK 67 ms 554 B Save Response

{} JSON Preview Visualize


```
{
    "date": "2026-04-20",
    "time": "18:00",
    "guestCount": 4,
    "availableTables": [
        {
            "id": 2,
            "capacity": 4,
            "status": "AVAILABLE"
        },
```


Admin APIs - View All Reservations

Reservation System > ... > Reservation ... > Happy Path > **Get All Reservation**

GET {{baseURL}} /admin/reservations/

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Headers Hide auto-generated headers

Key	Value	Descr...
Postman-Token	<calculated when request is sent>	
Host	<calculated when request is sent>	
User-Agent	PostmanRuntime/7.53.0	
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
Authorization	Bearer {{adminAccessToken}}	

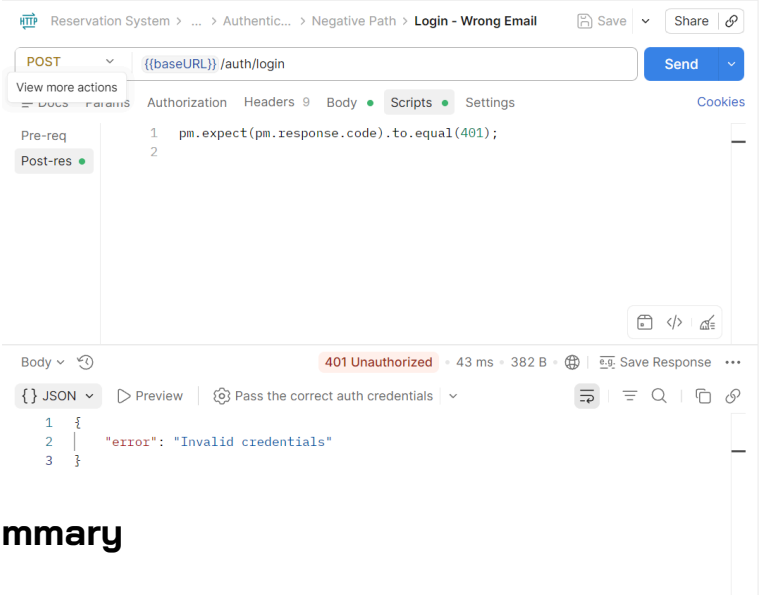
Body 200 OK 11 ms 2.6 KB Save Response

{} JSON Preview Visualize

View more actions


```
{
    "count": 8,
    "reservations": [
        {
            "id": 1776443479781,
            "tableId": 2,
            "date": "2026-04-18",
            "time": "18:00",
            "guestCount": 4,
            "status": "CONFIRMED",
```

Error Responses



7.3.3 Test Summary

After conducting comprehensive testing via **Postman**, the results indicate that the system successfully fulfills all core functional requirements, including **Authentication, Reservation, Table Management, Restaurant Info,** and **Notification.**

Overall Test Results

- **API Consistency:** The system responds correctly according to the predefined design.
- **HTTP Status Codes:** Proper implementation of status codes such as **200 (OK)**, **201 (Created)**, **400 (Bad Request)**, **401 (Unauthorized)**, **403 (Forbidden)**, and **404 (Not Found)**.
- **Role-Based Access Control (RBAC):** Permissions are correctly enforced for **Admin**, **Staff**, and **Customer** roles.
- **Data Validation:** Input validation for fields such as email, phone number, and seating capacity functions as expected.

Happy Path Results

- **Reservation Creation:** Successfully processed new bookings (**201 Created**).
- **Data Retrieval:** Accurate fetching of reservation lists (**200 OK**).
- **Management Operations:** Admins and Staff can manage tables and access global data effectively.
- **Admin Controls:** Restaurant information updates are restricted exclusively to Admin accounts.

Negative Path Results

- **401 Unauthorized:** Correctly returned when no token or role is provided.
- **403 Forbidden:** Correctly returned when a user attempts an action outside their role's permissions.
- **400 Bad Request:** Triggered by missing or invalid input data.
- **404 Not Found:** Returned when the requested resource does not exist.
- **409 Conflict:** Successfully identifies booking conflicts (e.g., double bookings)

System Limitations (Mock Data)

As the current system utilizes a **mock database**, the following limitations were observed:

- **Incomplete Data:** Some fields may return **null** (e.g., customerName, customerEmail).
- **Non-persistent Storage:** Data is volatile and resets upon server restart.
- **Data Relationships:** Some relational dependencies may not be fully integrated.

Conclusion

In summary, the API meets all primary requirements and has successfully passed both **Happy Path** and **Negative Path** testing. While the current version is functional, transitioning from mock data to a **production-grade database** is recommended to ensure data persistence and long-term stability

8. Automated Test

Automated testing is implemented using Playwright to validate key user workflows across Customer, Staff, and Admin roles. The framework focuses on end-to-end testing, role-based access, and core business flows.

8.1 Framework Design

Automation framework structure:

- `/tests` : test scenarios
- `/pages` : Page Object Model (POM)
- `/api` : API functions
- `/testdata` : test data

8.2 Test Data

User Login Data

- Email
- Password

Reservation Data

- Date
- Time
- Number of guests
- Special request

Restaurant Data

- Name
- Opening time
- Closing time
- Phone number
- Email

8.3 Test Scenarios

- FR-02 / TC-01: Valid login (Customer)
- FR-03 / TC-02: View available tables
- FR-04 / TC-02: Create reservation
- FR-05 / TC-02: View reservations
- FR-06 / TC-01: Edit reservation
- FR-07 / TC-01: Cancel reservation
- FR-08 / TC-01: Admin notifications
- FR-09 / TC-03: Staff update table status
- FR-10 / TC-02: Multiple reservations
- FR-12 / TC-01: Update restaurant info

8.3 Test Script

https://github.com/isandwish/sw-testing-project/tree/main/automated-test

8.4 Test Result

TC ID	Test Case	Expected Result	Actual Result	Status
TC-01	Valid login (Customer)	Login success	Login success	Passed
TC-02	View available tables	Show available tables	Correct display	Passed
TC-03	Create reservation	Reservation created	Created successfully	Passed
TC-04	View reservations	Show user reservations	Correct data shown	Passed
TC-05	Edit reservation	Update successful	Updated successfully	Passed

TC-06	Cancel reservation	Cancel success	Cancelled successfully	Passed
TC-07	Admin notification	Notification received	Received	Passed
TC-08	Staff update table status	Status updated	Updated to OUT_OF_SERVICE	Passed
TC-09	Multiple reservations	Allow multiple bookings	Working correctly	Passed
TC-10	Update restaurant info	Data updated	Updated successfully	Passed

8.5 Summary

All automated test cases passed successfully. Core functionalities including authentication, reservation management, role-based access, and admin configuration are working as expected. The system is stable within the tested scope.

9. Performance Testing

9.1 Test Scenario

- 1000 concurrent users
- Mixed workload: login, reservation, table query

9.2 Performance Criteria

- Login ⩽ 1 second
- Reservation ⩽ 2 seconds
- Table API ⩽ 1 second
- Error rate < 1%
- Duration: 1 hour stability test

9.3 Tool

- Apache JMeter

9.4 Performance Test Script

[Project Performance Test.jmx]

9.5 Performance Testing Results Summary

To evaluate the system's ability to handle real-world traffic based on the requirements, two main user flows were simulated:

1. **Customer Booking Flow:** Logging in, checking availability, creating, updating, and deleting reservations.
2. **Admin Management Flow:** Logging in, viewing reservations, checking tables, and updating restaurant info.

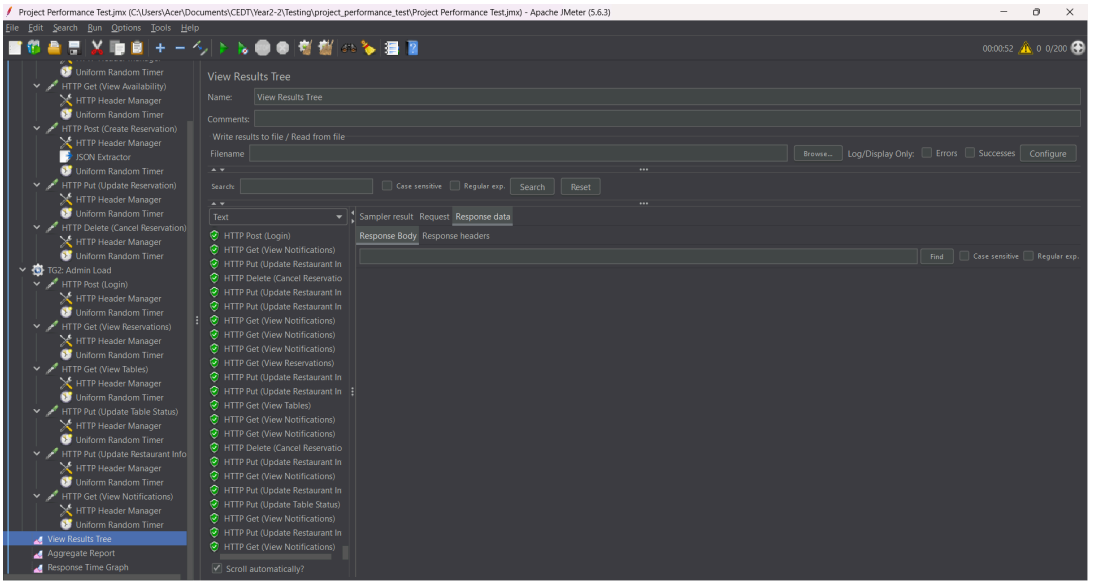
9.6 Requirements vs. Actual Results

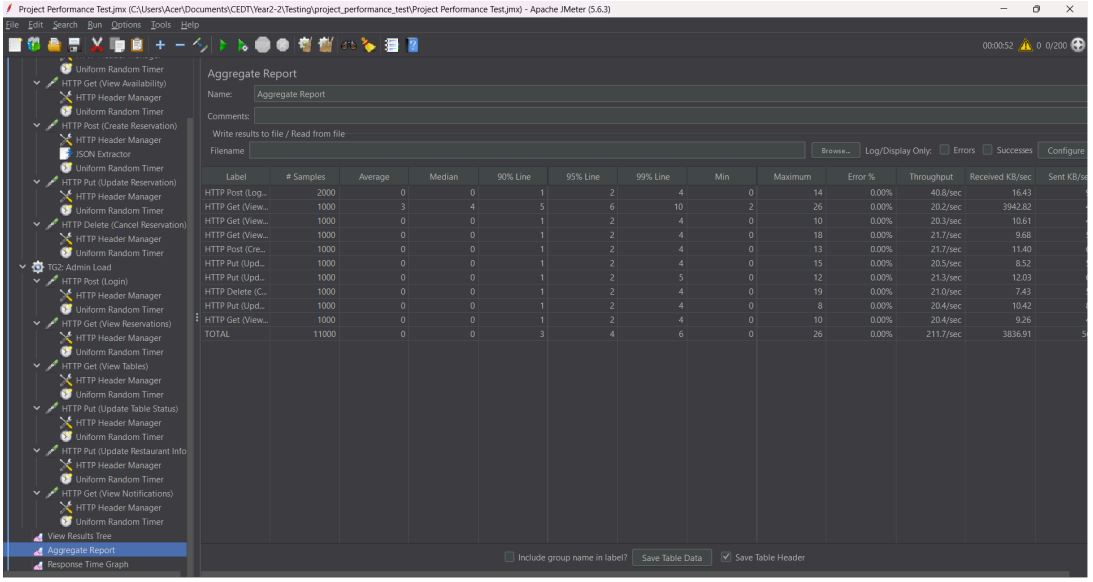
Based on the evaluation of the Apache JMeter results against the 4 specified requirements, the detailed findings are as follows:

9.6.1 Concurrent Users: The system must support 1,000 concurrent users.

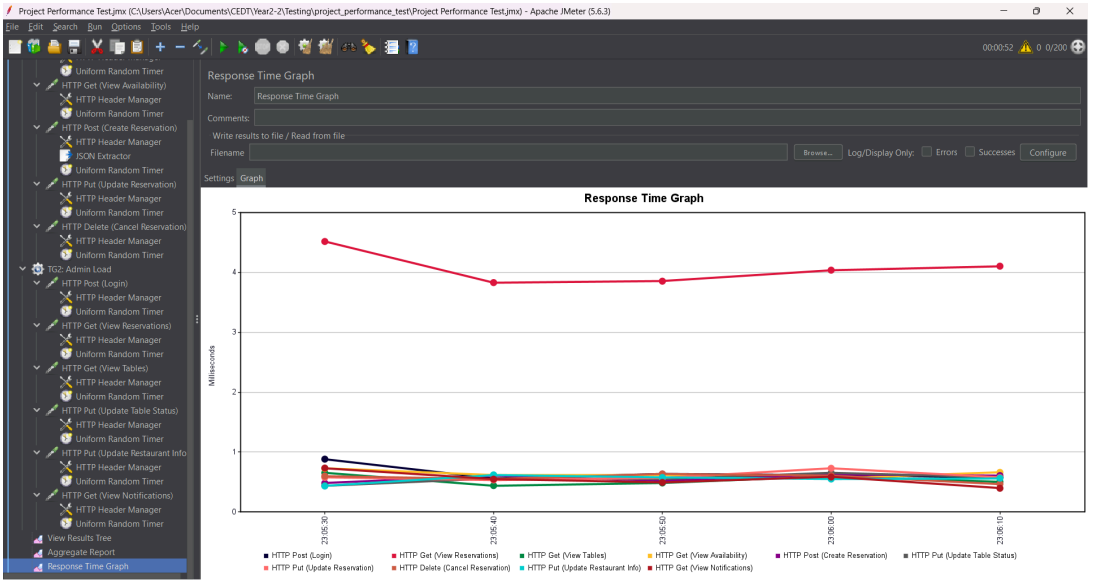
- **Test Result:** Passed flawlessly. The testing was divided into 4 phases to evaluate the system's capacity under various stress levels:

- **Phase 1 - Load Test at 100 Users (Baseline):**
  The response time graph remained completely flat and at a very low level. The system managed all requests rapidly and seamlessly without any interruptions, representing a highly comfortable traffic volume for the server.

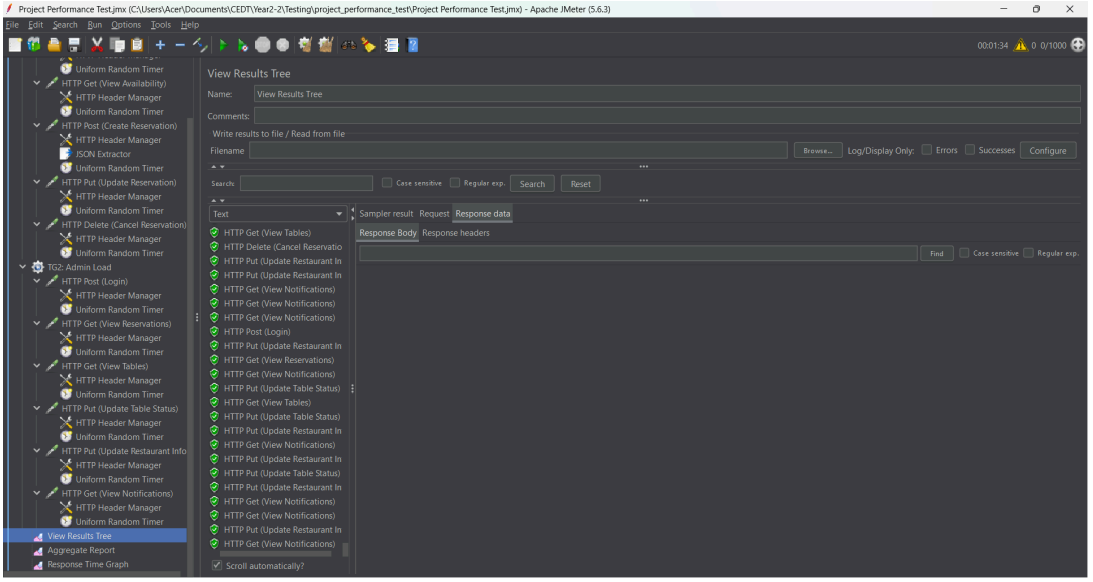


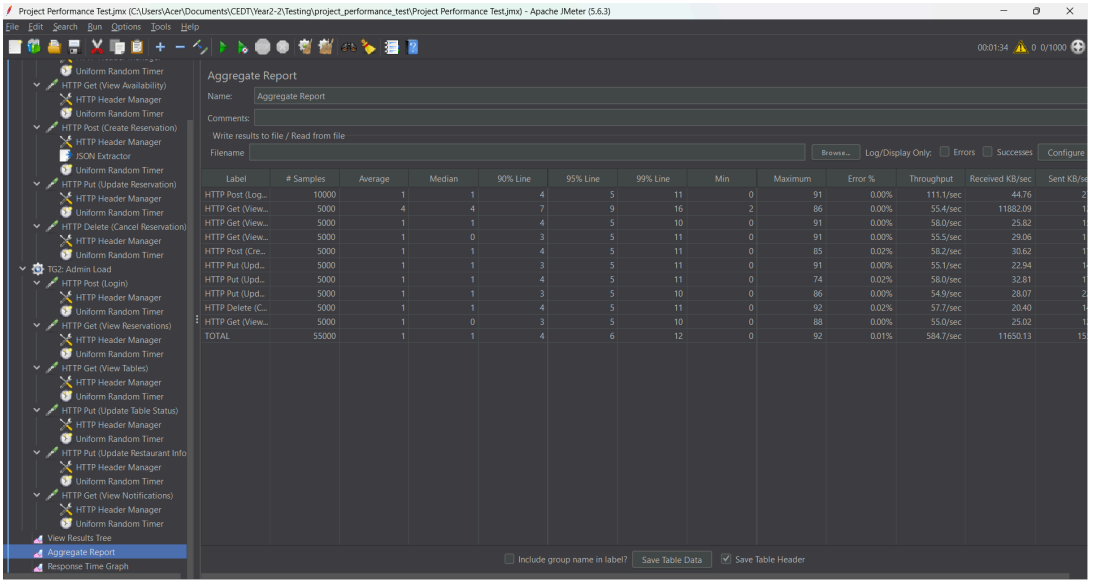


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
HTTP Post (Log...	2000	0	0	1	2	4	0	14	0.00%	40.8/sec	16.43	[illegible]
HTTP Get (View...	1000	3	4	5	6	10	2	26	0.00%	20.2/sec	3942.82	[illegible]
HTTP Get (View...	1000	0	0	1	2	4	0	10	0.00%	20.3/sec	10.61	[illegible]
HTTP Get (View...	1000	0	0	1	2	4	0	18	0.00%	21.7/sec	9.68	[illegible]
HTTP Post (Cre...	1000	0	0	1	2	4	0	13	0.00%	21.7/sec	11.40	[illegible]
HTTP Put (Upd...	1000	0	0	1	2	4	0	15	0.00%	20.5/sec	8.52	[illegible]
HTTP Put (Upd...	1000	0	0	1	2	5	0	12	0.00%	21.3/sec	12.03	[illegible]
HTTP Delete (C...	1000	0	0	1	2	4	0	19	0.00%	21.0/sec	7.43	[illegible]
HTTP Put (Upd...	1000	0	0	1	2	4	0	8	0.00%	20.4/sec	10.42	[illegible]
HTTP Get (View...	1000	0	0	1	2	4	0	10	0.00%	20.4/sec	9.26	[illegible]
TOTAL	11000	0	0	3	4	6	0	26	0.00%	211.7/sec	3836.91	[illegible]

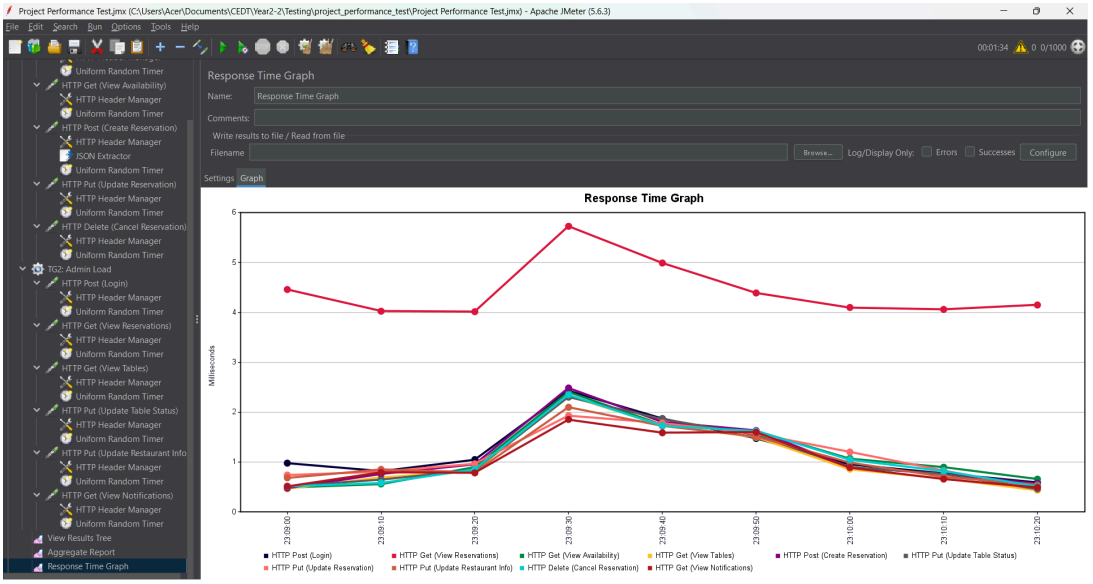


- **Phase 2 - Load Test at 500 Users (Medium Load):**
  As the number of concurrent users increased, the graph displayed occasional spikes, specifically during write operations (such as creating or updating reservations). However, the response times quickly dropped back to normal levels. This indicates highly effective queue management without accumulating bottlenecks.

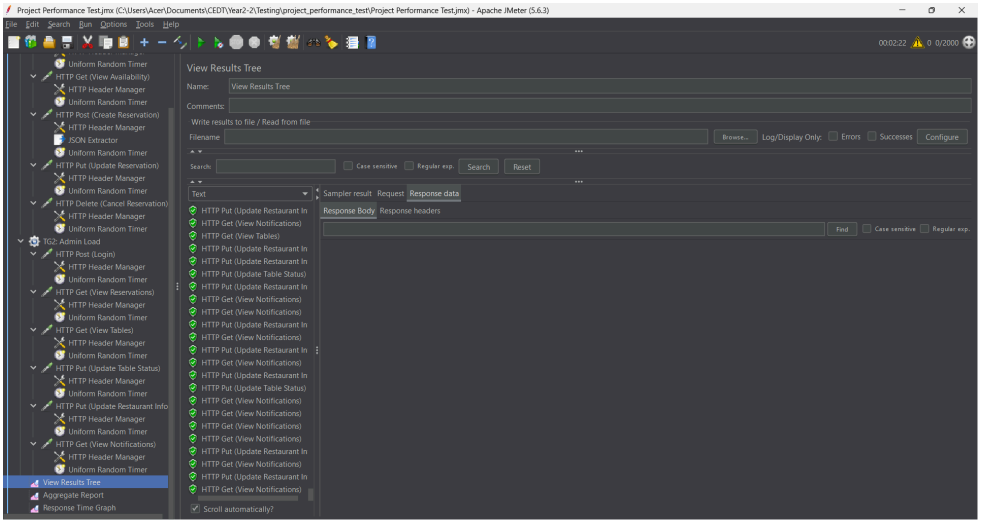


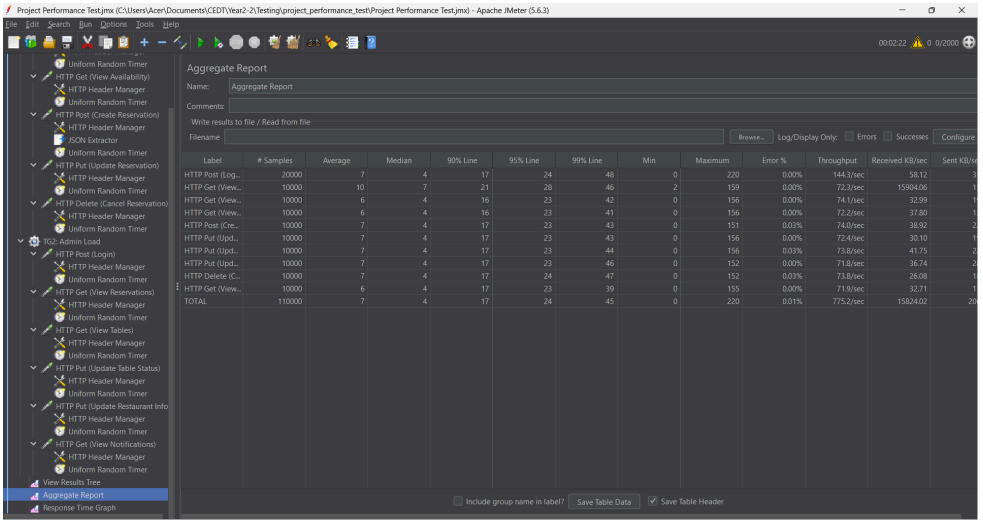


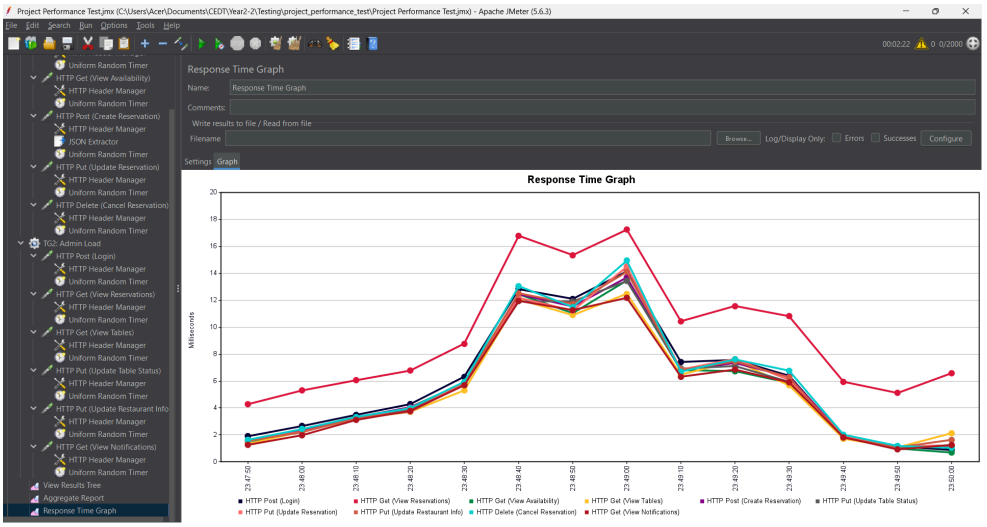
Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/se
HTTP Post (Log...	10000	1	1	4	5	11	0	91	0.00%	111.1/sec	44.76	2
HTTP Get (View...	5000	4	4	7	9	16	2	86	0.00%	55.4/sec	11882.09	1
HTTP Get (View...	5000	1	1	4	5	10	0	91	0.00%	58.0/sec	25.82	1
HTTP Get (View...	5000	1	0	3	5	11	0	91	0.00%	55.5/sec	29.06	1
HTTP Post (Cre...	5000	1	1	4	5	11	0	85	0.02%	58.2/sec	30.62	1
HTTP Put (Upd...	5000	1	1	3	5	11	0	91	0.00%	55.1/sec	22.94	1
HTTP Put (Upd...	5000	1	1	4	5	11	0	74	0.02%	58.0/sec	32.81	1
HTTP Put (Upd...	5000	1	1	3	5	10	0	86	0.00%	54.9/sec	28.07	2
HTTP Delete (C...	5000	1	1	4	5	11	0	92	0.02%	57.7/sec	20.40	1
HTTP Get (View...	5000	1	0	3	5	10	0	88	0.00%	55.0/sec	25.02	1
TOTAL	55000	1	1	4	6	12	0	92	0.01%	584.7/sec	11650.13	15



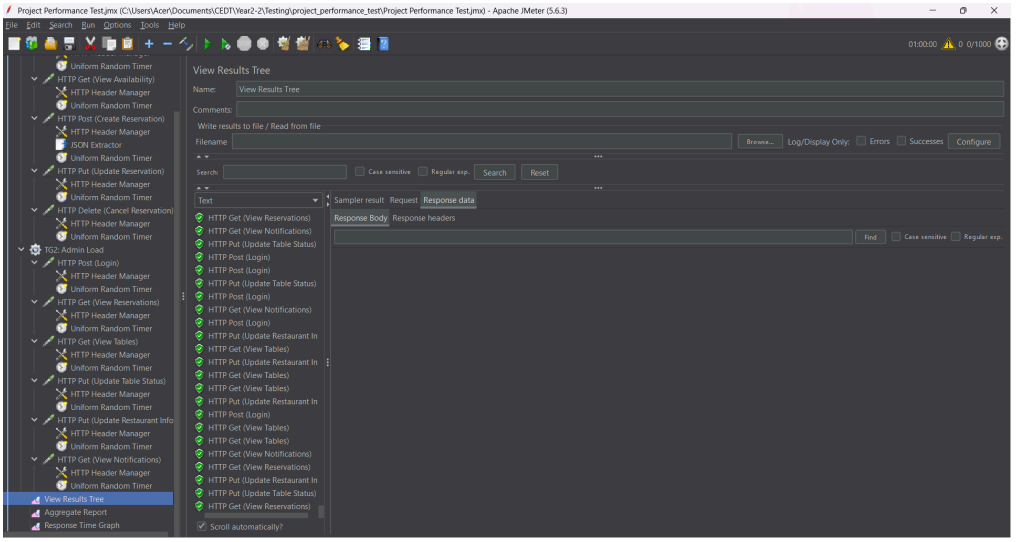
- **Phase 3 - Load Test at 1,000 Users (Maximum Required Load):**
  This phase applied the heaviest stress to the system. While response times fluctuated and wait times increased noticeably compared to previous rounds, the most crucial finding was that **zero failures occurred**. Every single request was processed to completion, definitively proving that the system can support 1,000 concurrent users without crashing.

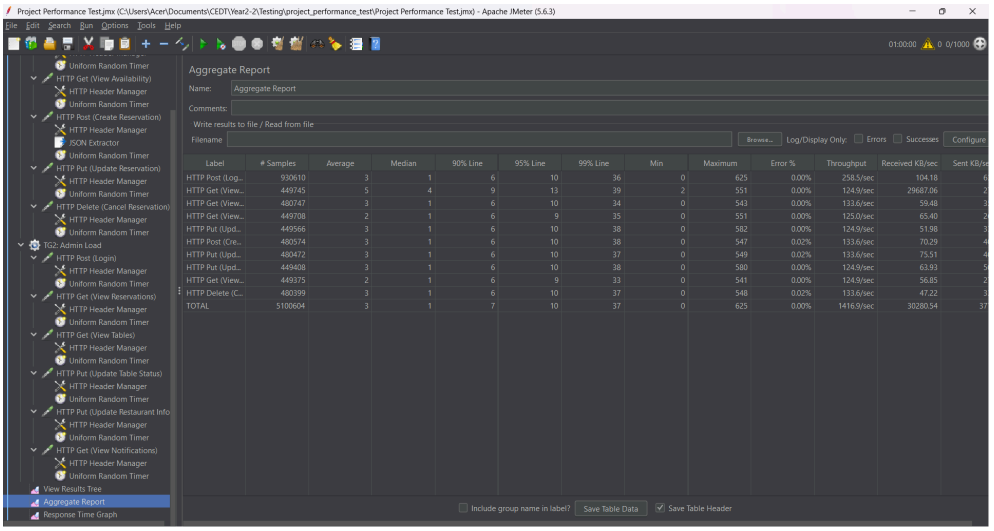


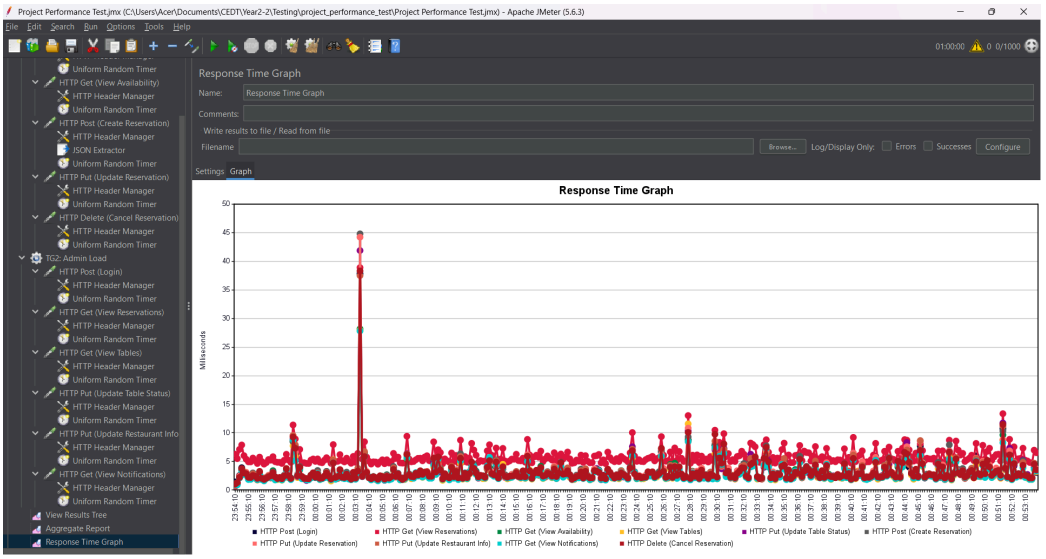




- **Phase 4 - Soak Test at 500 Users for 1 Continuous Hour (Endurance Test):** Conducted to observe system behavior under sustained load. The results showed that response times remained consistent throughout the entire hour. The system exhibited no signs of performance degradation, fatigue, or sluggishness over time.

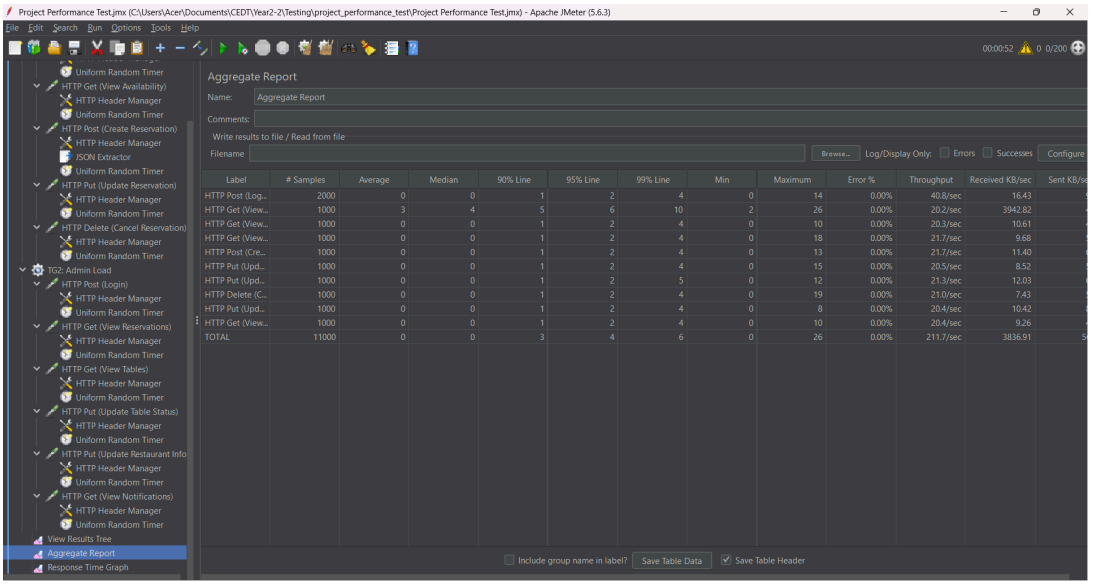


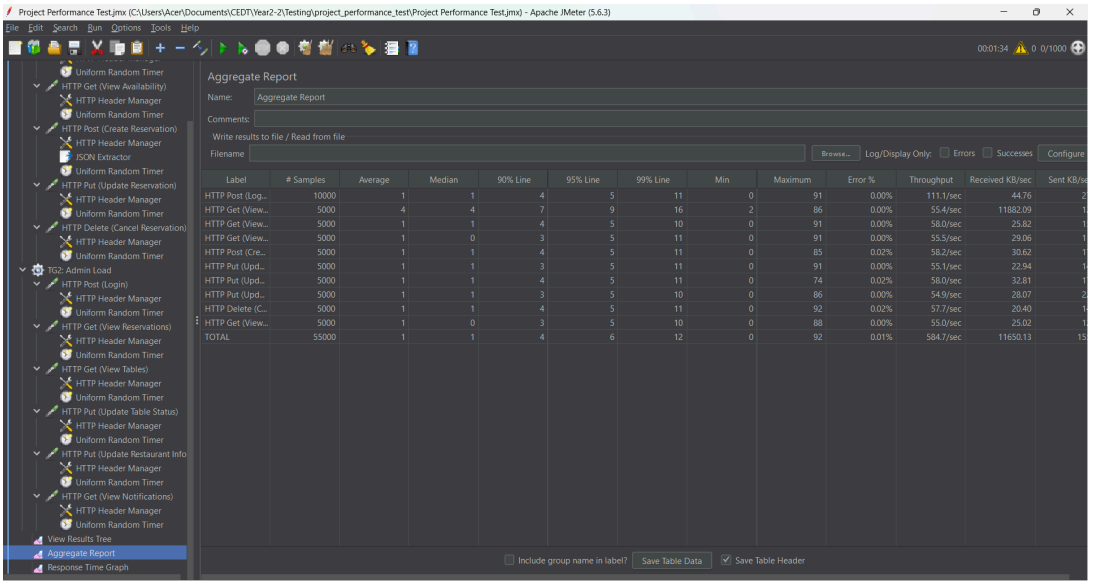


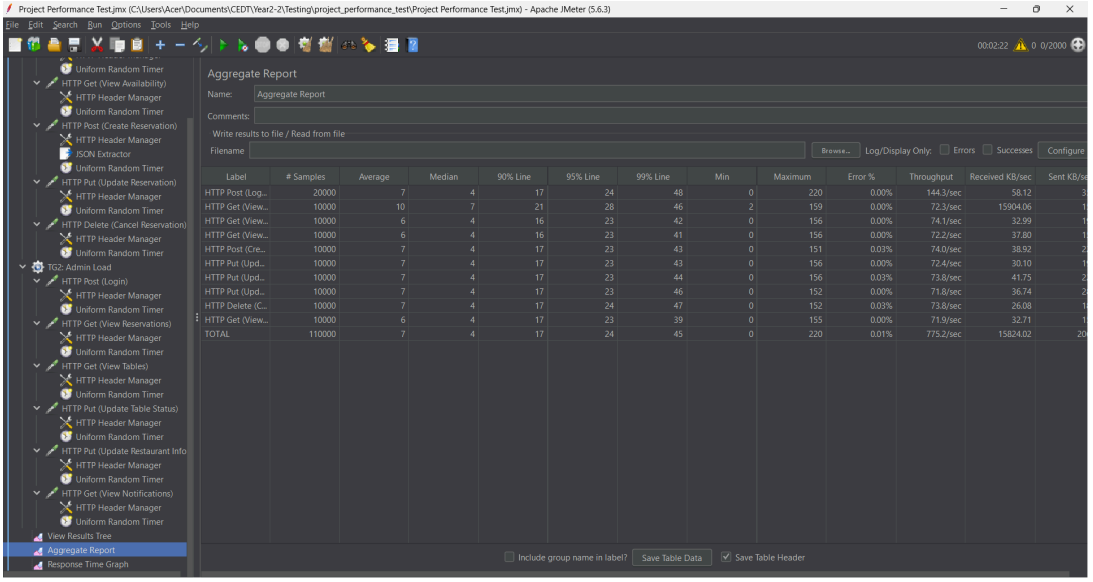


9.6.2 Error Rate: The error rate must be less than 1%.

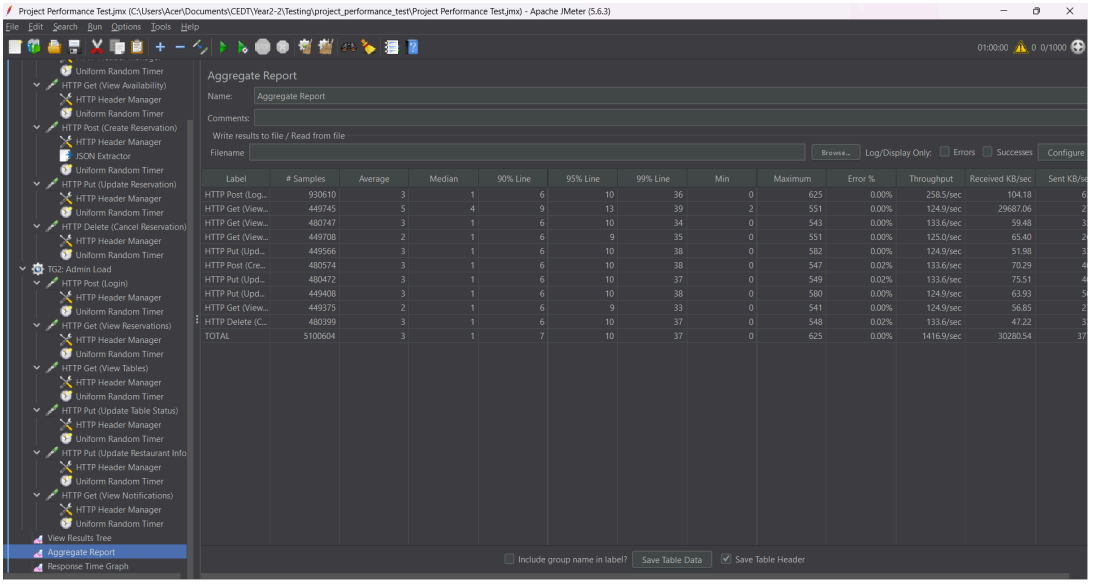
- **Test Result:** Significantly exceeded expectations. According to the Aggregate Report from the heaviest load test (1,000 users), the error rate was exactly **0.00% - 0.01%** across all transactions. This means no booking data was lost, no pages froze, and absolutely no 500 Internal Server Errors were returned by the server.







Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/se
HTTP Post (Log...	20000	7	4	17	24	48	0	220	0.00%	144.3/sec	58.12	3
HTTP Get (View...	10000	10	7	21	28	46	2	159	0.00%	72.3/sec	15904.06	1
HTTP Get (View...	10000	6	4	16	23	42	0	156	0.00%	74.1/sec	32.99	1
HTTP Get (View...	10000	6	4	16	23	41	0	156	0.00%	72.2/sec	37.80	1
HTTP Post (Cre...	10000	7	4	17	23	43	0	151	0.03%	74.0/sec	38.92	2
HTTP Put (Upd...	10000	7	4	17	23	43	0	156	0.00%	72.4/sec	30.10	1
HTTP Put (Upd...	10000	7	4	17	23	44	0	156	0.03%	73.8/sec	41.75	2
HTTP Put (Upd...	10000	7	4	17	23	46	0	152	0.00%	71.8/sec	36.74	2
HTTP Delete (C...	10000	7	4	17	24	47	0	152	0.03%	73.8/sec	26.08	1
HTTP Get (View...	10000	6	4	17	23	39	0	155	0.00%	71.9/sec	32.71	1
TOTAL	110000	7	4	17	24	45	0	220	0.01%	775.2/sec	15824.02	20

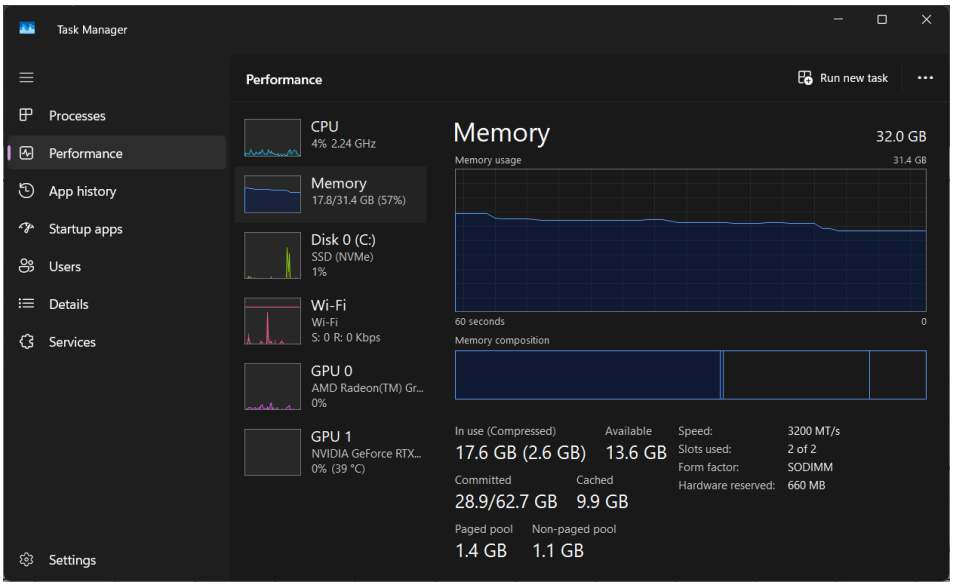


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/se
HTTP Post (Log...	930610	3	1	6	10	36	0	625	0.00%	258.5/sec	104.18	6
HTTP Get (View...	449745	5	4	9	13	39	2	551	0.00%	124.9/sec	29687.06	2
HTTP Get (View...	480747	3	1	6	10	34	0	543	0.00%	133.6/sec	59.48	3
HTTP Get (View...	449708	2	1	6	9	35	0	551	0.00%	125.0/sec	65.40	2
HTTP Put (Upd...	449566	3	1	6	10	38	0	582	0.00%	124.9/sec	51.98	3
HTTP Post (Cre...	480574	3	1	6	10	38	0	547	0.02%	133.6/sec	70.29	4
HTTP Put (Upd...	480472	3	1	6	10	37	0	549	0.02%	133.6/sec	75.51	4
HTTP Put (Upd...	449408	3	1	6	10	38	0	580	0.00%	124.9/sec	63.93	5
HTTP Get (View...	449375	2	1	6	9	33	0	541	0.00%	124.9/sec	56.85	2
HTTP Delete (C...	480399	3	1	6	10	37	0	548	0.02%	133.6/sec	47.22	3
TOTAL	5100604	3	1	7	10	37	0	625	0.00%	1416.9/sec	30280.54	37

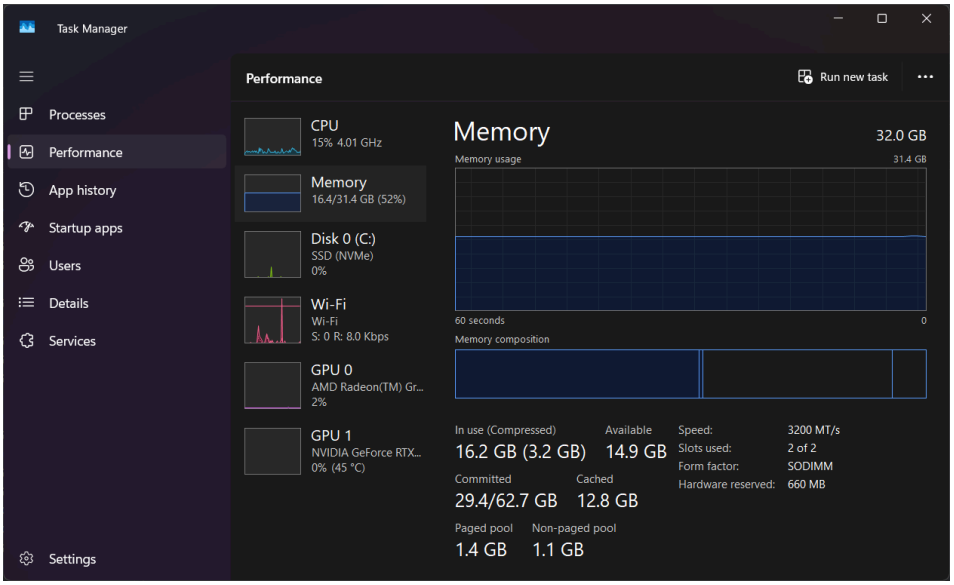
9.6.3 Duration: The system must run smoothly for 1 hour (Soak Test).

- **Test Result:** Passed without any defects. A continuous 1-hour Soak Test was performed to verify server stability and check for potential memory leaks. The resource utilization monitored via Task Manager showed the following:

- *Before the test:* CPU usage at 4% and RAM at 17.6 GB.



- *After the test:* CPU usage at 15% and RAM at 16.2 GB.

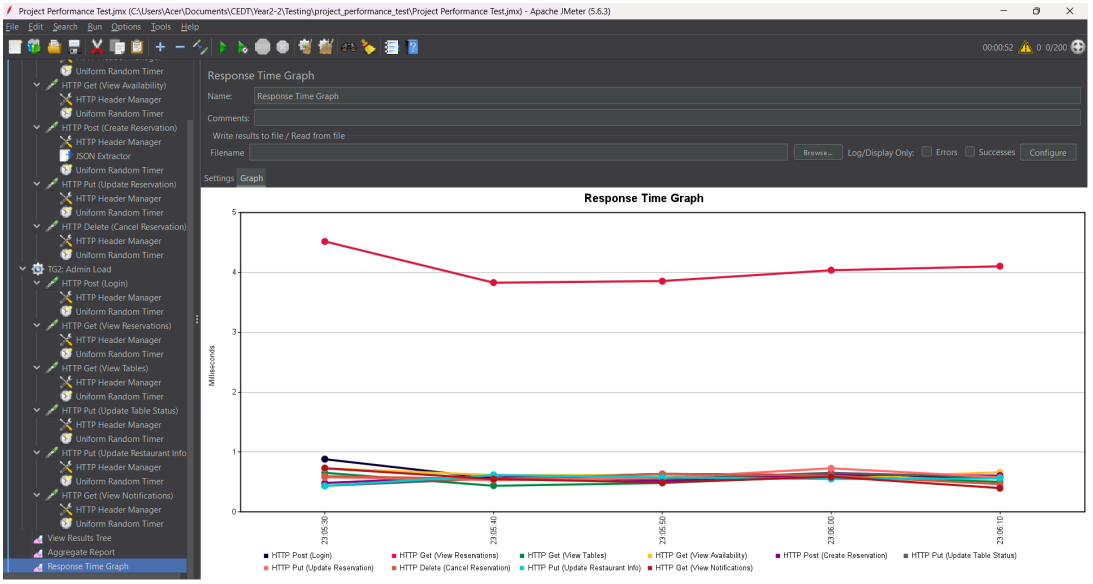


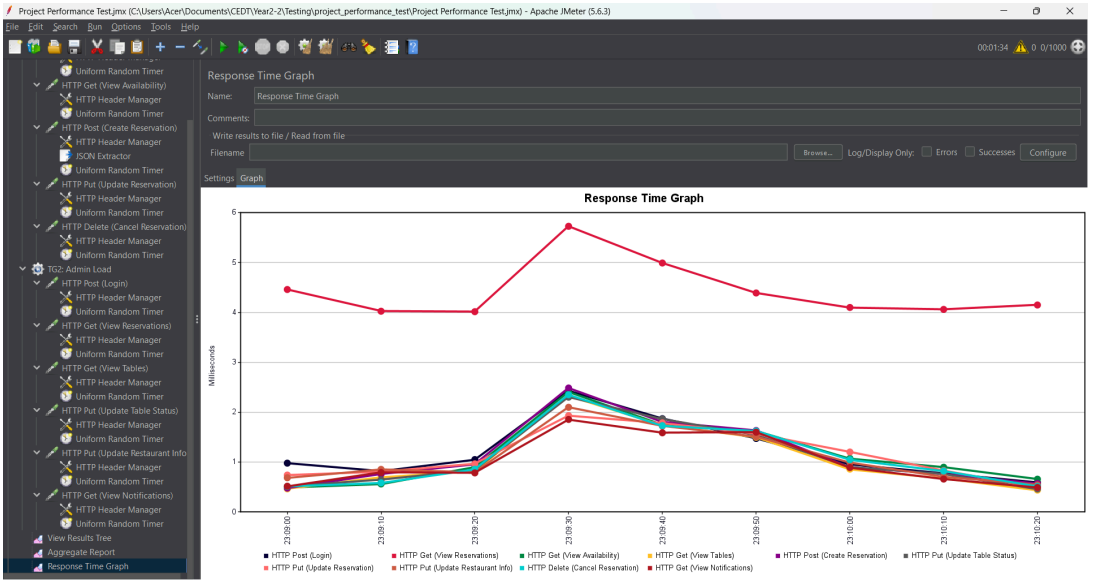
- **Conclusion:** There was no significant negative difference in CPU and RAM usage before and after the test. The system successfully released hardware resources back to normal states, confirming the absence of memory leaks or frozen background processes.

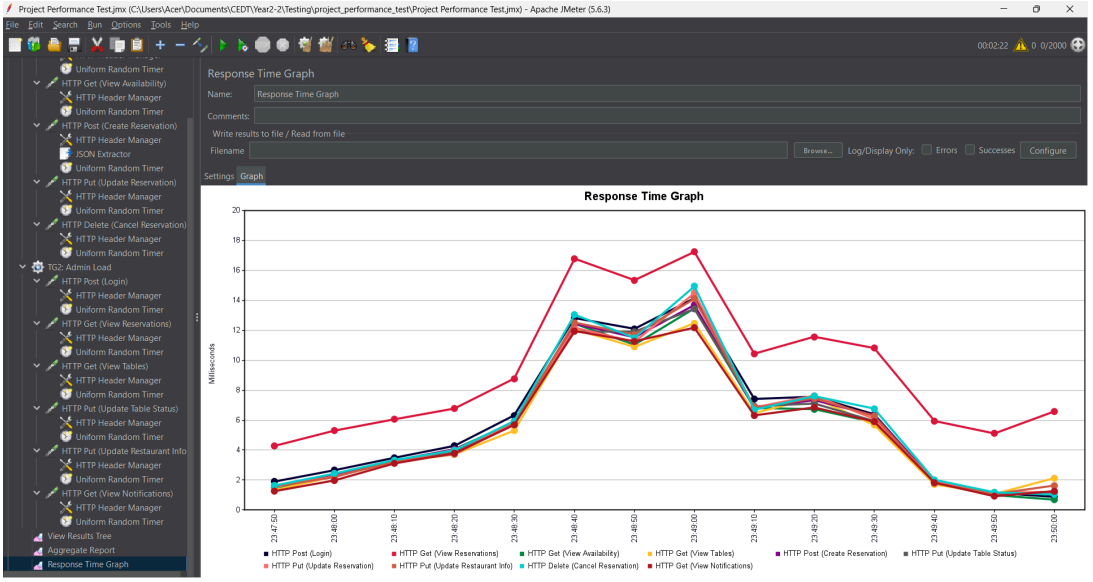
9.6.4 Response Time Requirement

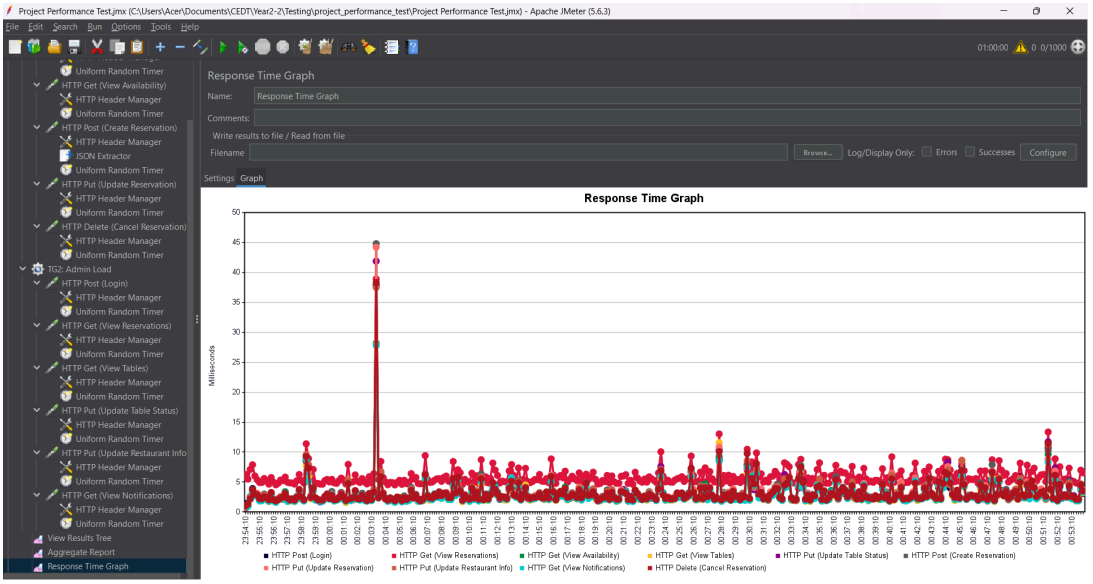
- **Test Result:** Across all load tests, initial read operations maintained very low response times. When the system entered database write operations (e.g., Create/Update Reservation), natural spikes in response time occurred. However, the system consistently demonstrated the ability to **instantly recover and clear the queue**, bringing the graph back down to acceptable levels. This proves the

system possesses a highly efficient request-handling mechanism.









9.7 Summary

The restaurant reservation management system is entirely **Production-Ready**. It perfectly accommodates 1,000 concurrent users with a 0% error rate and maintains excellent hardware resource stability during prolonged continuous usage. All performance testing criteria have been successfully achieved across all dimensions.

10. Security Testing

10.1 Scope

Security testing was performed to ensure the system protects user data and prevents unauthorized access. The scope includes:

- JWT token validation
- Token expiration handling
- Unauthorized access prevention
- Input validation (SQL Injection / XSS basic checks)
- Role-based access control (Customer / Staff / Admin)
- Login brute-force protection (basic)
- Password policy validation (OWASP A07 – Identification and Authentication Failures)

10.2 Security Test Approach

Security testing was conducted to identify vulnerabilities and verify that the system properly protects sensitive data and user accounts.

The following testing methods were used:

- Authentication testing (login, register, password rules)
- Token-based security testing (JWT validation and expiration)
- Authorization testing (role and access control checks)
- Input validation testing (SQL Injection and XSS payloads)
- Business logic validation (invalid inputs such as negative values or past dates)
- Brute-force simulation using repeated login attempts

Testing was performed using manual testing and security tools (e.g., OWASP ZAP) to simulate real attack scenarios.

10.3 Security Test Result

10.3.1 Password Policy

- OWASP Top 10 (2025)

A07 Identification and Authentication Failures

- ระดับความเสี่ยง (CVSS v3.1)

Medium (5.3)

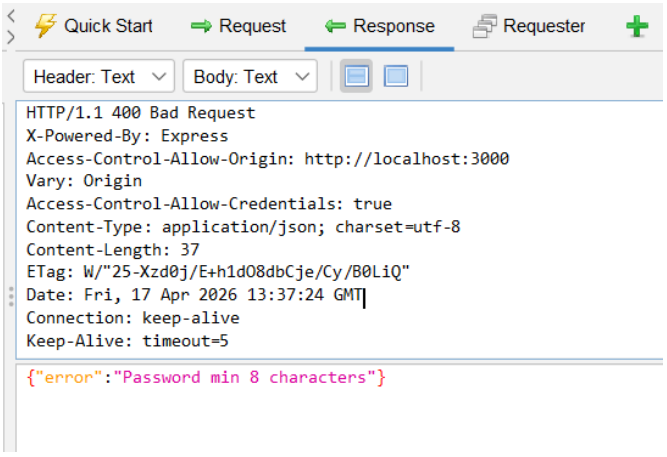
Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถยิง Request ผ่าน Internet ได้โดยตรง
Attack Complexity (AC)	Low	ความซับซ้อนต่ำมาก เพียงแค่พิมพ์รหัสผ่าน ผ่านหน้าเว็บหรือ API
Privileges Required (PR)	None	ไม่จำเป็นต้องมีสิทธิ์หรือบัญชีเข้าใช้งานมาก่อน
User Interaction (UI)	None	ไม่ต้องอาศัยInteract จาก User อื่น
Scope (S)	Unchanged	ขอบเขตจำกัดอยู่แค่บัญชีที่ถูกเดารหัสผ่านได้ ไม่ลุกลามไปยังระบบอื่น
Confidentiality (C)	Low	เข้าถึงข้อมูลได้เฉพาะบัญชีที่ถูกเดารหัสผ่านเท่านั้น
Integrity (I)	None	ไม่ส่งผลให้ Database ถูกแก้ไข
Availability (A)	None	ระบบยังทำงานได้ตามปกติ

- ขั้นตอนการทดสอบ
    1. สร้าง HTTP POST Request ไปที่ Endpoint /auth/register
    2. กำหนด Content-Type: application/json ใน HTTP Header
    3. ส่งข้อมูล (Payload) โดยใช้รหัสผ่านที่ไม่ตรงตามมาตรฐานความปลอดภัย ตามเงื่อนไข

ได้แก่ ยาว 8-20 ตัวอักษร, รหัสผ่านมีตัวพิมพ์ใหญ่อย่างน้อยหนึ่งตัวอักษร, รหัสผ่านมีตัวเลขอย่างน้อยหนึ่งตัวอักษร

- ผลการทดสอบที่คาดหวัง ระบบปฏิเสธและแจ้งสถานะ HTTP/1.1 400 Bad Request
- ผลการทดสอบ

ทดสอบรหัสผ่านน้อยกว่า 8 ตัวอักษร



ทดสอบรหัสผ่านน้อยกว่า 20 ตัวอักษร

Quick Start | Request | Response | Requester

Header: Text | Body: Text

```
Access-Control-Allow-Origin: http://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 38
ETag: W/"26-NbhLfxp5+IHpkGLb0HM+m/FDx2M"
Date: Fri, 17 Apr 2026 13:40:31 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
{"error":"Password max 20 characters"}
```

Quick Start | Request | Response | Requester

Header: Text | Body: Text

```
POST http://localhost:5000/api/auth/register HTTP/1.1
host: localhost:5000
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:149.0) Gecko/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
```

```
{
  "fullName": "Test Short",
  "email": "test1@test.com",
  "password": "Pass123456789Worddddd!",
  "phoneNumber": "0812345678"
}
```

ทดสอบรหัสผ่านไม่มีตัวเลข

Quick Start | Request | Response | Requester

Header: Text | Body: Text

```
POST http://localhost:5000/api/auth/register HTTP/1.1
host: localhost:5000
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:149.0) Gecko/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
```

```
{
  "fullName": "Test No Number",
  "email": "test3@test.com",
  "password": "Password!!!",
  "phoneNumber": "0812345678"
}
```

Quick Start | Request | Response | Requester

Header: Text | Body: Text

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 32
ETag: W/"20-IG7D9La85lKPYnvqmFDoFUm/0jQ"
Date: Fri, 17 Apr 2026 13:41:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
{"error":"Password need number"}
```

ทดสอบรหัสผ่านไม่มีตัวพิมพ์ใหญ่

Quick Start | Request | Response | Requester

Header: Text | Body: Text

```
POST http://localhost:5000/api/auth/register HTTP/1.1
host: localhost:5000
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:149.0) Gecko/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
```

```
{
  "fullName": "Test Lowercase",
  "email": "test2@test.com",
  "password": "password123!",
  "phoneNumber": "0812345678"
}
```

Quick Start | Request | Response | Requester

Header: Text | Body: Text

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 42
ETag: W/"2a-gEiqbaha1rUsgyIV/2dFbgbUZNw"
Date: Fri, 17 Apr 2026 13:43:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
{"error":"Password need uppercase letter"}
```

- แนวทางการแก้ไข ใช้ Regular Expression (Regex) เพื่อตรวจสอบความซับซ้อนของรหัสผ่านที่ฝั่งเซิร์ฟเวอร์

10.3.2 Login Attempt Limitation

- OWASP Top 10 (2025)

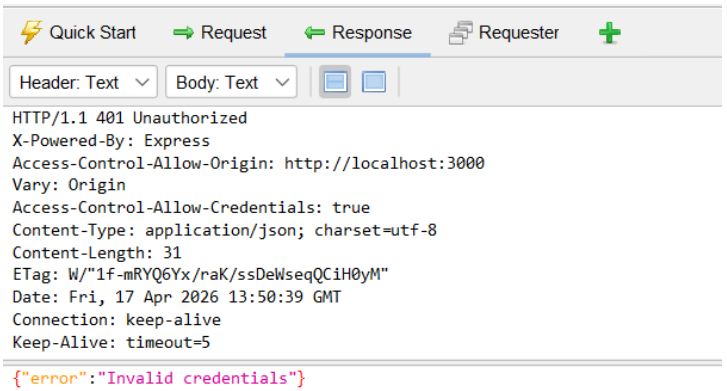
A07 Identification and Authentication Failures

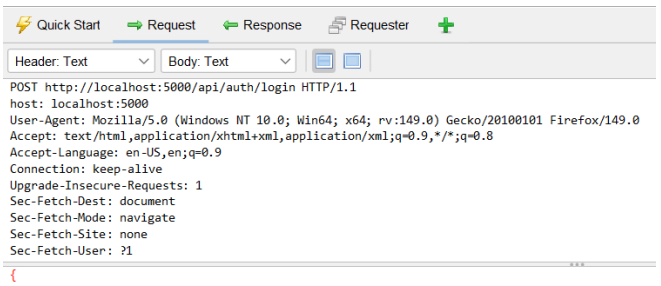
- ระดับความเสี่ยง (CVSS v3.1)

High (7.3)

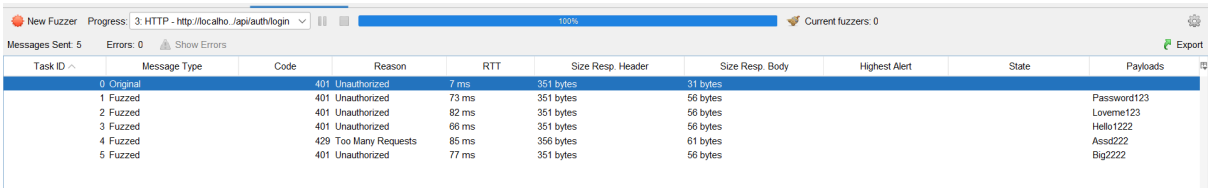
Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถยิง Request ผ่าน Internet ได้โดยตรง
Attack Complexity (AC)	Low	สามารถใช้โปรแกรมอัตโนมัติ เช่น ZAP Fuzzer ยิงรหัสผ่านซ้ำ ๆ ได้
Privileges Required (PR)	None	ไม่จำเป็นต้องมีสิทธิ์หรือบัญชีเข้าใช้งานมาก่อน
User Interaction (UI)	None	ไม่ต้องอาศัยInteract จาก User อื่น
Scope (S)	Unchanged	ผลกระทบจำกัดอยู่ภายในระบบ Authentication ของ application
Confidentiality (C)	Low	เข้าถึงข้อมูลได้เฉพาะบัญชีที่ถูกเดารหัสผ่านเท่านั้น
Integrity (I)	Low	สามารถแอบดูข้อมูลและแก้ไขข้อมูลส่วนตัวของ account รายนั้นได้
Availability (A)	Low	การยิง Request จำนวนมากอาจทำให้เซิร์ฟเวอร์ตอบสนองช้าลง

- ขั้นตอนการทดสอบ (Test Steps)
    1. สร้าง HTTP POST Request ไปที่ Endpoint /auth/login
    2. ใช้เครื่องมือ Fuzzer ในโปรแกรม OWASP ZAP เพื่อจำลองการส่ง Request ด้วยรหัสผ่านที่ผิดพลาดมากกว่า 5 ครั้งติดต่อกัน
- ผลการทดสอบที่คาดหวัง ระบบระงับการเข้าถึงชั่วคราวและตอบกลับด้วยสถานะ HTTP/1.1 429 Too Many Requests
- ผลการทดสอบ

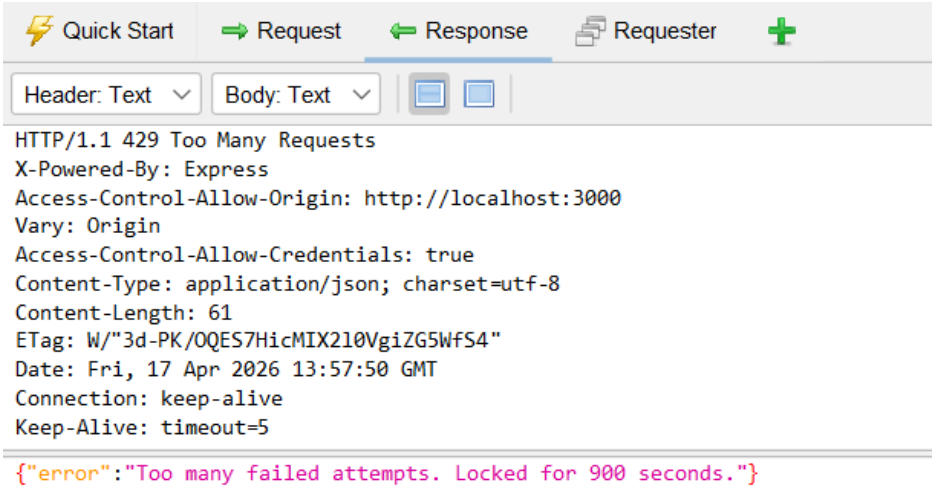




ส่งผิดครั้งที่ 5 ขึ้นไป



Task ID	Message Type	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	State	Payloads
0	Original	401	Unauthorized	7 ms	351 bytes	31 bytes			
1	Fuzzed	401	Unauthorized	73 ms	351 bytes	56 bytes			Password123
2	Fuzzed	401	Unauthorized	82 ms	351 bytes	56 bytes			Loveme123
3	Fuzzed	401	Unauthorized	66 ms	351 bytes	56 bytes			Hello1222
4	Fuzzed	429	Too Many Requests	85 ms	356 bytes	61 bytes			Assd222
5	Fuzzed	401	Unauthorized	77 ms	351 bytes	56 bytes			Big2222



ตารางสรุปผลการทดลอง

request No.	Payload	HTTP Status	response
1-5	(incorrect passwords)	401 Unauthorized	Invalid credentials
6	Loveme123	429 Too many Requests	Acoount temporarily

- แนวทางการแก้ไข นำระบบ Rate Limiting และ Account Lockout มาใช้งาน หากผิดเกิน 5 ครั้ง ให้ระงับชั่วคราว 15 นาที

10.3.3 Token Expiration

- OWASP Top 10 (2025)

A07 Identification and Authentication Failures

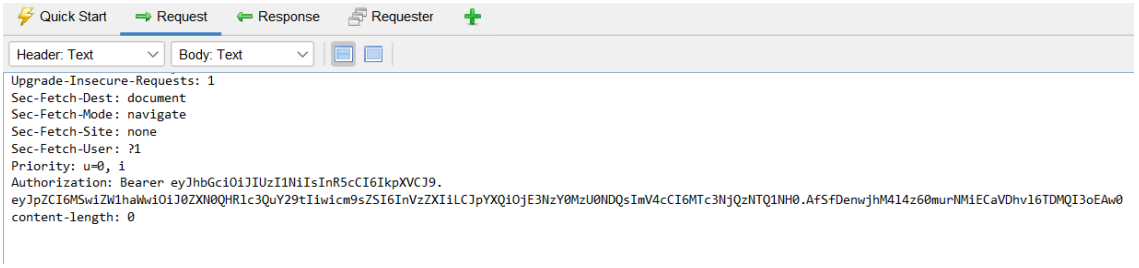
- ระดับความเสี่ยง (CVSS v3.1)

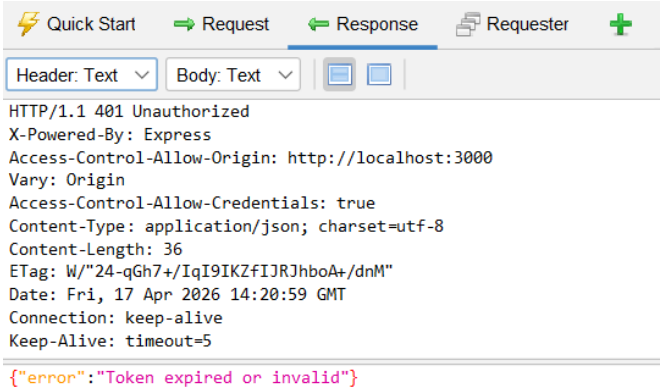
High (8.1)

Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถยิง Request จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	High	ความซับซ้อนสูง เพราะผู้โจมตีต้องหาทาง ได้ Token ที่หมดอายุไปแล้วของ user มาให้ได้ก่อน
Privileges Required (PR)	None	ไม่จำเป็นต้องมีสิทธิ์หรือบัญชีเข้าใช้งานมาก่อน
User Interaction (UI)	None	เมื่อได้ Token มาแล้ว สามารถนำไปใช้ยิง API ได้ทันที
Scope (S)	Unchanged	ขอบเขตการโจมตีจำกัดอยู่แค่ในระบบเดิม ไม่ได้ลุกลามข้ามไปแอปพลิเคชันอื่น
Confidentiality (C)	High	ถ้าโจมตีสำเร็จจะสามารถเรียกใช้ API เพื่อดึงข้อมูลส่วนตัวทั้งหมดของเหยื่อออกมาดูได้
Integrity (I)	High	ผู้โจมตีสามารถสวมสิทธิ์เพื่อทำธุรกรรมปลอมแปลงได้ทุกอย่าง
Availability (A)	High	Attacker สามารถเข้าไปยกเลิกการจองโต๊ะทั้งหมดของเหยื่อได้ ทำให้สูญเสียสิทธิ์ในการเข้าถึงบริการ หรือ Attacker อาจใช้ Token นี้ยิง Request รัวๆ จนบัญชีของเหยื่อถูกระบบแบน

- ขั้นตอนการทดสอบ (Test Steps):
    1. จำลองการสร้าง JWT โดยแก้ไขค่าระยะเวลาหมดอายุ (exp) ให้เป็นเวลาในอดีต (ไม่ได้เปลี่ยน Signing key)
    2. ส่ง HTTP GET Request โดยแนบค่า Token ที่หมดอายุลงใน Header Authorization: Bearer <Expired_Token>
- ผลการทดสอบที่คาดหวัง: ระบบปฏิเสธการเข้าถึงและตอบกลับด้วยสถานะ HTTP/1.1 401 Unauthorized

- ผลการทดสอบ





เมื่อส่ง Request ด้วย JWT ที่ระบุค่า exp เป็นเวลาที่ผ่านมาแล้ว ระบบทำการปฏิเสธคำร้องขอทันที โดยคืนค่าสถานะ 401 Unauthorized

- แนวทางการแก้ไข กำหนดอายุ Access Token ให้สั้น (15-30 นาที) คู่ไปกับ Refresh Token และตรวจสอบค่า exp ผ่าน Middleware ทุกครั้ง

10.3.4 Input Validation Security

10.3.4.1 SQL Injection Testing

- OWASP Top 10 (2025)

A03 Injection

- ระดับความเสี่ยง (CVSS v3.1)

High(8.8)

Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถยิง Request จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	Low	โจมตีได้ง่ายใช้ String พื้นฐานในการทดสอบ
Privileges Required (PR)	None	ไม่จำเป็นต้องมีสิทธิ์หรือบัญชีเข้าใช้งานมาก่อน

Metric	Value	Reason
User Interaction (UI)	None	สามารถยิง api ผ่าน payload ได้โดยตรง
Scope (S)	Unchanged	ขอบเขตการโจมตีจำกัดอยู่แค่ในระบบเดิม ไม่ได้ลุกลามข้ามไปแอปพลิเคชันอื่น
Confidentiality (C)	High	สามารถดึงข้อมูลออกมาได้ทั้งฐานข้อมูล
Integrity (I)	High	ข้อมูลในฐานข้อมูลอาจถูกลบหรือแก้ไข
Availability (A)	High	อาจทำให้ฐานข้อมูลล่มหรือลบตารางทิ้ง

- ขั้นตอนการทดสอบ (Test Steps)
    1. ส่งพารามิเตอร์อีเมลเป็นคำสั่ง SQL อันตราย เช่น ' OR 1=1 – ,' and 1=0 union all select...
- ผลการทดสอบที่คาดหวัง ระบบต้องตรวจสอบ Input และปฏิเสธ Request ด้วย HTTP 400 Bad Request
- ผลการทดสอบ

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 39
ETag: W/"27-Qiz9AYQ6fZ3tWWm35jRShRxws9Q"
Date: Fri, 17 Apr 2026 14:57:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Invalid characters detected"}
```

New Fuzzer Progress: 4: HTTP - http://localho.../api/auth/login 100% Current fuzzers: 0

Messages Sent: 13 Errors: 0 Show Errors Export

Task ID	Message Type	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	State	Payloads
1	Fuzzed	400	Bad Request	17 ms	350 bytes	39 bytes			' OR 1=1 --
2	Fuzzed	400	Bad Request	9 ms	350 bytes	39 bytes			' OR '1'='1
3	Fuzzed	400	Bad Request	23 ms	350 bytes	39 bytes			' OR 1=1#
4	Fuzzed	400	Bad Request	18 ms	350 bytes	39 bytes			admin' --
5	Fuzzed	400	Bad Request	14 ms	350 bytes	39 bytes			admin' #
6	Fuzzed	400	Bad Request	20 ms	350 bytes	39 bytes			' OR 'x'='x
7	Fuzzed	400	Bad Request	30 ms	381 bytes	1,144 bytes			" OR "a"="a
8	Fuzzed	400	Bad Request	33 ms	350 bytes	39 bytes			'
9	Fuzzed	400	Bad Request	36 ms	381 bytes	1,144 bytes		Reflected	"
10	Fuzzed	401	Unauthorized	38 ms	351 bytes	31 bytes			'
11	Fuzzed	400	Bad Request	40 ms	350 bytes	39 bytes			') OR ('1'='1
12	Fuzzed	400	Bad Request	42 ms	350 bytes	39 bytes			'; SELECT 1/0 --

- แนวทางการแก้ไข

1.ใช้ Parameterized Queries (Prepared Statements) หรือ ORM (Object-Relational Mapping) ในการเชื่อมต่อและดึงข้อมูลจากฐานข้อมูลเสมอ

2.ห้ามนำค่าจากผู้ใช้งาน (Input) มาต่อ String (Concatenation) เพื่อสร้างคำสั่ง SQL โดยตรง

10.3.4.2 Cross-Site Scripting (XSS) Testing

- OWASP Top 10 (2025)

A03 Injection

- ระดับความเสี่ยง (CVSS v3.1)

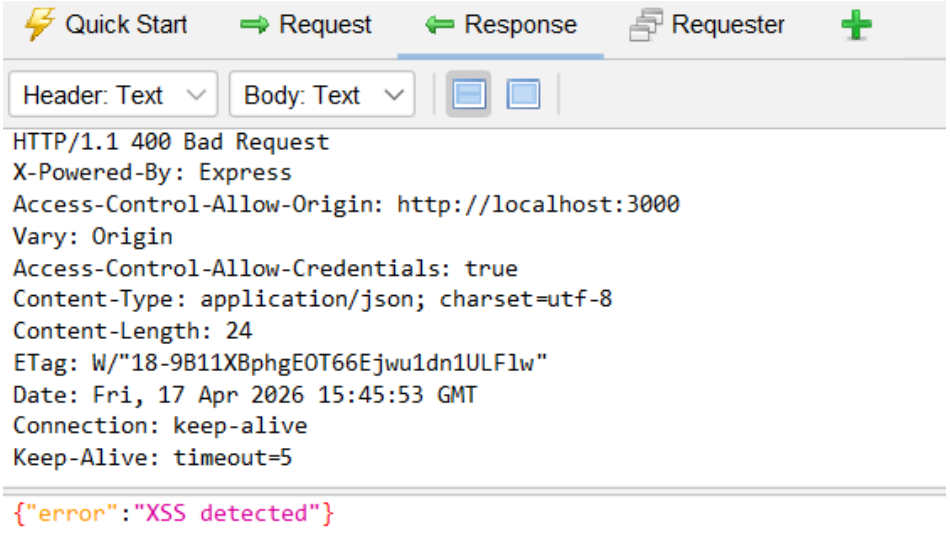
Medium (6.1)

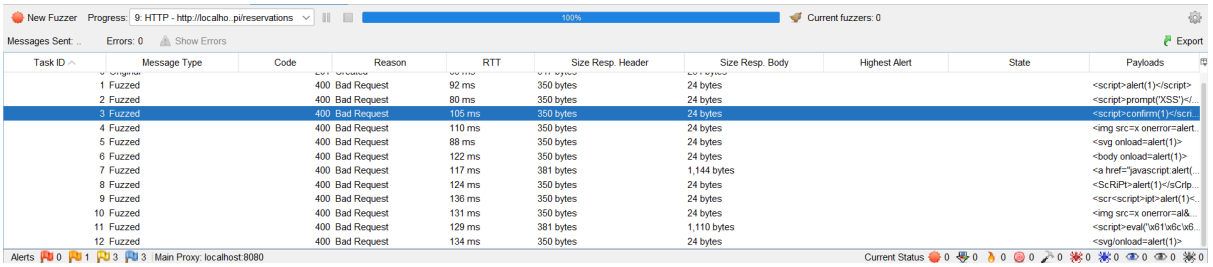
Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถฝัง Script จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	Low	โจมตีได้ง่ายใช้สคริปต์ alert พื้นฐานในการทดสอบ
Privileges Required (PR)	None	ไม่จำเป็นต้องมีสิทธิ์หรือบัญชีเข้าใช้งานมาก่อน
User Interaction (UI)	Required	ต้องมีผู้ใช้คนอื่น (เช่น Admin) มาดูข้อมูลนี้เนื่องจากข้อนี้เป็น Stored XSS
Scope (S)	Changed	สคริปต์ทำงานใน Browser ของคนอื่น
Confidentiality (C)	Low	แอบขโมย Cookie หรือ Session ได้
Integrity (I)	Low	ปรับเปลี่ยนหน้าตาเว็บชั่วคราว
Availability (A)	None	ไม่กระทบเซิร์ฟเวอร์

- ขั้นตอนการทดสอบ (Test Steps)
    1. ใส่สคริปต์ในช่องคำขอพิเศษ เช่น <script>alert(1)</script>
- ผลการทดสอบที่คาดหวัง

ระบบต้องล้างรหัสสคริปต์ (Sanitize) หรือปฏิเสธด้วย HTTP 400

- ผลการทดสอบ





- แนวทางการแก้ไข

1.ทำ Output Encoding ทุกครั้งก่อนนำข้อมูลที่รับจากผู้ใช้ไปแสดงผลบนหน้าเว็บ (เปลี่ยน < เป็น <)

2.ตรวจสอบและทำความสะอาด (Sanitization) ข้อมูลฝั่ง Backend ก่อนบันทึกลงฐานข้อมูล

3. กำหนด Content Security Policy (CSP) ใน HTTP Header เพื่อป้องกันการรันสคริปต์จากภายนอก

10.3.4.3 Business Logic

- OWASP Top 10 (2025)

  A06 Insecure Design

- ระดับความเสี่ยง (CVSS v3.1)

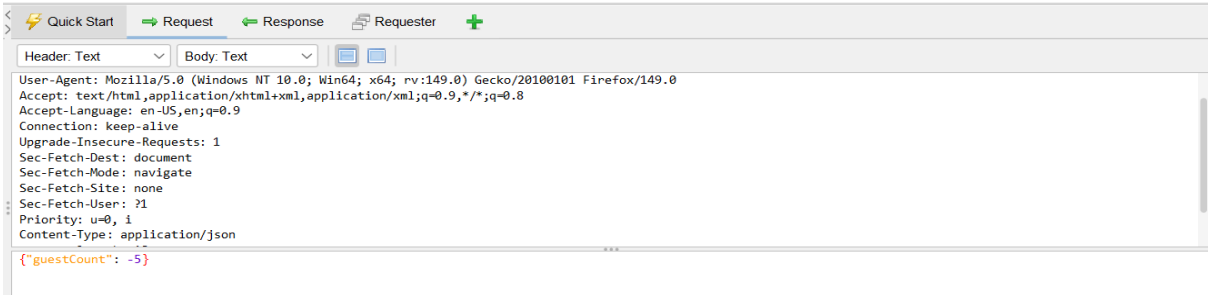
Medium (5.3)

Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถปรับเปลี่ยนค่าตัวเลขหรือวันที่ใน API จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง

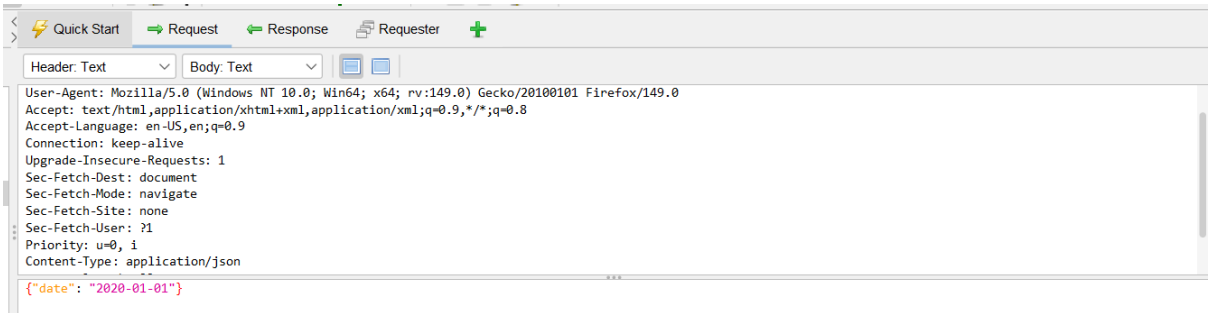
Metric	Value	Reason
Attack Complexity (AC)	Low	โจมตีได้ง่ายใช้การแก้ไขค่าใน JSON Payload
Privileges Required (PR)	Low	ต้องมีบัญชีผู้ใช้ถึงจะทำการโจมตีได้
User Interaction (UI)	None	สามารถยิง api ผ่าน payload ได้โดยตรง
Scope (S)	Unchanged	ขอบเขตการโจมตีจำกัดอยู่แค่ในระบบเดิม ไม่ได้ลุกลามข้ามไปแอปพลิเคชันอื่น
Confidentiality (C)	Low	อาจเห็นโครงสร้างไฟล์จาก Error Message
Integrity (I)	None	ไม่กระทบความถูกต้องเชิงโครงสร้าง
Availability (A)	None	ไม่กระทบการทำงานหลัก

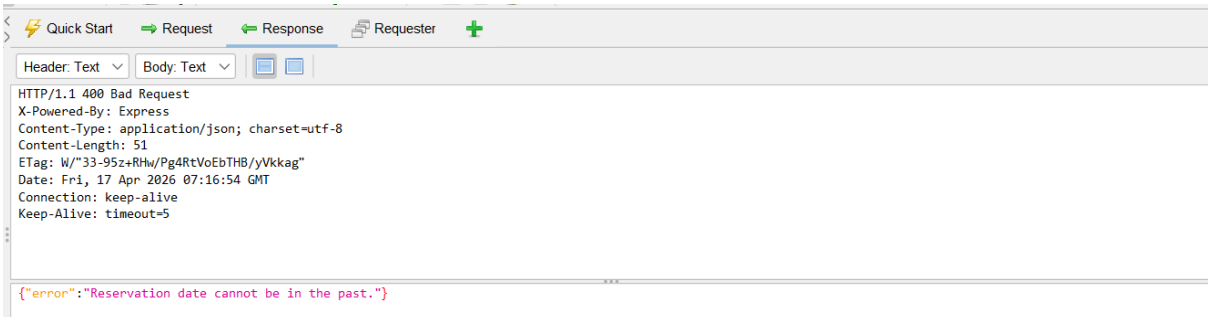
- ขั้นตอนการทดสอบ (Test Steps)
    1. ส่ง Request จองโต๊ะโดยใส่จำนวนคนติดลบ หรือวันที่ในอดีต หรือส่ง JSON ที่ผิดไวยากรณ์
- ผลการทดสอบ

guest count จำนวนติดลบ



จองวันที่ในอดีต





- ผลการทดสอบที่คาดหวัง (Expected Result)

ระบบปฏิเสธด้วย HTTP 400 และแจ้ง Error (ไม่โชว์ Stack Trace)

- แนวทางการแก้ไข
    1. ตรวจสอบ Logic ที่ฝั่ง Backend
    2. ปิดการแสดงผล Debug/Stack Trace ใน Environment ที่เป็น Production โดยให้แสดงผลเฉพาะ Generic Error Message

ตารางสรุปผล

Attack type	Payload	HTTP Status	Response Message
SQL Injection	' OR 1=1 -- ' OR '1'='1 ' OR 1=1# admin' -- admin' # ' OR 'x'='x " OR "a"="a ' " ` ') OR ('1'='1 '; SELECT 1/0 --	400 Bad Request	Security Alert: Invalid characters detected!

Cross-Site Scripting	<script>alert(1)</script> <script>prompt('XSS')</script> <script>confirm(1)</script> <img src=x onerror=alert('XSS')> <svg onload=alert(1)> <body onload=alert(1)> <input autofocus onfocus=alert(1)> <a href="javascript:alert(1)">Click Me</a> <ScRiPt>alert(1)</sCrIpT> <scr<script>ipt>alert(1)</script> <img src=x onerror=alert(1)> <script>eval('\x61\x6c\x65\x72\x74(1)')</script> <svg/onload=alert(1)>	400 Bad Request	Xss Detected
Business Logic	{"guestCount": -5}, {"date": "2020-01-01"}	400 Bad Request	Invalid guest count. Must be between 1 and 10 Reservation date cannot be in the past.

10.3.5 API Security

10.3.5.1 Vertical Privilege Escalation

- OWASP Top 10 (2025)

A01 Broken Access Control (BOLA)

- ระดับความเสี่ยง (CVSS v3.1)

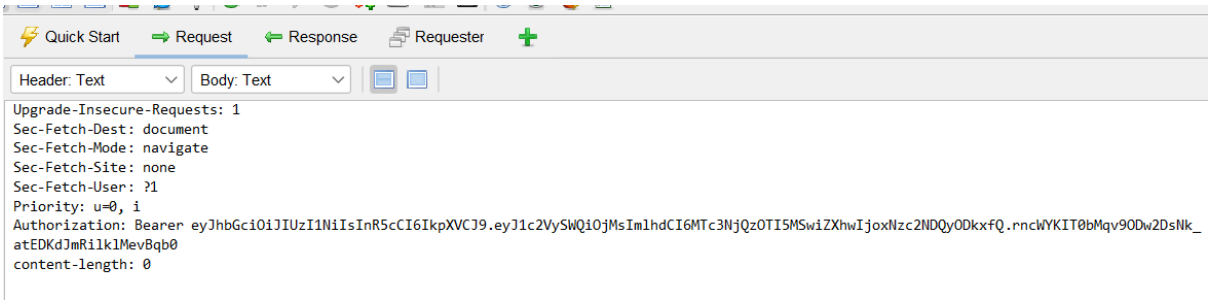
High (8.8)

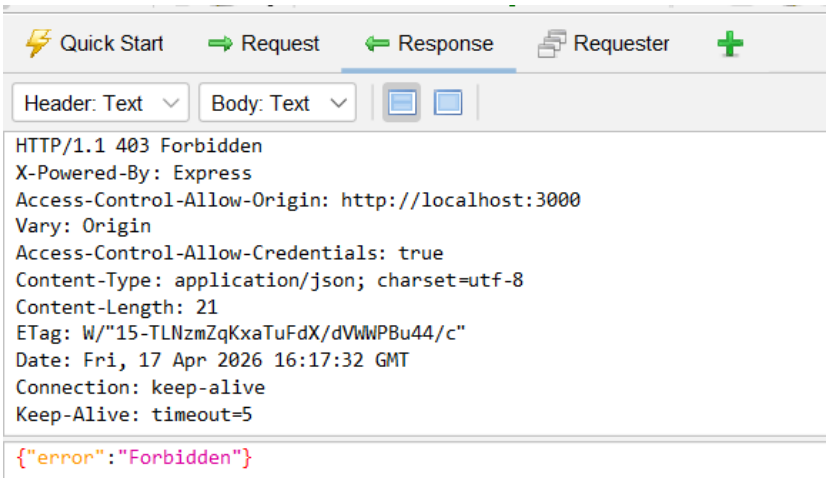
Metric	Value	Reason

Attack Vector (AV)	Network	ผู้โจมตีสามารถยิง Request จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	Low	โจมตีผ่านอินเทอร์เน็ตได้ง่าย เพียงแค่เปลี่ยน URL ไปยังเส้นทางของ Admin
Privileges Required (PR)	Low	attacker จำเป็นต้องมีสิทธิ์ระดับเริ่มต้น (เช่น ต้องสมัครสมาชิกและ Login เป็น Customer ก่อน) ถึงจะได้ Token มาใช้ข้ามสิทธิ์
User Interaction (UI)	None	ไม่ต้องอาศัย Interact จาก User อื่น
Scope (S)	Unchanged	ไม่ได้ลุกลามไปยัง Server ตัวอื่น
Confidentiality (C)	High	ถ้าโจมตีสำเร็จ จะสามารถควบคุมระบบได้เสมือนเป็นผู้ดูแลระบบ จัดการข้อมูลลูกค้าทั้งหมดได้
Integrity (I)	High	ถ้าโจมตีสำเร็จ จะสามารถแก้ไขการตั้งค่าหรือรายละเอียดของร้านได้
Availability (A)	High	ถ้าโจมตีสำเร็จ จะสามารถลบข้อมูลสำคัญทิ้งได้

- ขั้นตอนการทดสอบ (Test Steps)
    1. ใช้ Access Token ของระดับลูกค้า (Customer) เข้าถึง Endpoint ของ Admin เช่น /admin/reservations
- ผลการทดสอบที่คาดหวัง ระบบตอบกลับด้วยสถานะ HTTP/1.1 403 Forbidden
- ผลการทดสอบ

จากการทดสอบโดยใช้ Access Token ของผู้ใช้งานระดับ Customer ส่ง Request ไปยัง Endpoint /admin/reservations ระบบมีการตรวจสอบสิทธิ์ และพบว่าผู้ใช้ไม่มีสิทธิ์เข้าถึงฟังก์ชันของผู้ดูแลระบบ จึงทำการปฏิเสธคำร้องขอทันที





- แนวทางการแก้ไข บังคับใช้การตรวจสอบ Role ของผู้ใช้งานผ่าน Middleware สำหรับทุก Admin Endpoint

10.3.5.2 Horizontal Privilege Escalation

- OWASP Top 10 (2025)

A01 Broken Access Control (BOLA)

- ระดับความเสี่ยง (CVSS v3.1)

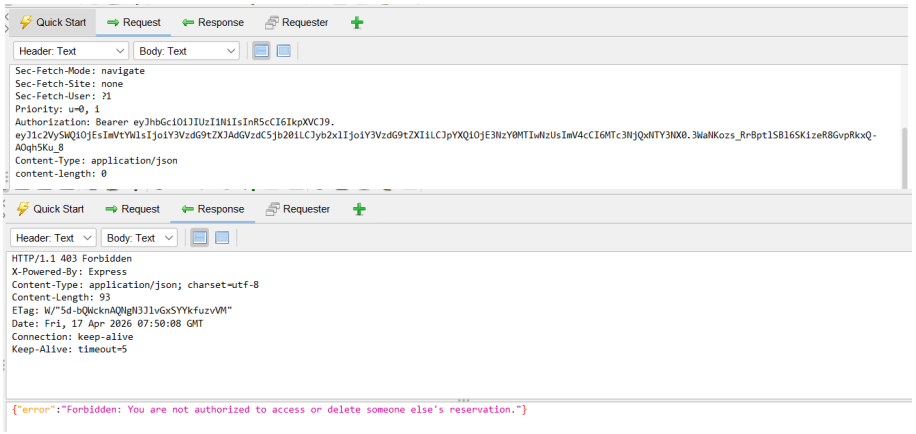
High (7.1)

Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถแก้ไข ID ใน URL API จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	Low	โจมตีได้ง่ายโดยเดาตัวเลข reservation id จากของตัวเอง
Privileges Required (PR)	Low	ต้องมีสิทธิ์ Customer ของตนเอง
User Interaction (UI)	None	สามารถยิง api ผ่าน payload ได้โดยตรง
Scope (S)	Unchanged	ขอบเขตการโจมตีจำกัดอยู่แค่ในระบบเดิม ไม่ได้ลุกลามข้ามไปแอปพลิเคชันอื่น
Confidentiality (C)	Low	สามารถเห็นข้อมูลลูกค้าคนอื่น
Integrity (I)	High	สามารถแอบยกเลิกการจองของคนอื่นได้

- ขั้นตอนการทดสอบ (Test Steps)
    1. ใช้ Token ของ A สั่งลบ reservationId ของ B
- ผลการทดสอบที่คาดหวัง (Expected Result)

ระบบต้องตอบกลับด้วย HTTP 403 หรือ 404

- ผลการทดสอบ



- แนวทางการแก้ไข
    - ก่อนที่จะให้ดู แก้ไข หรือลบข้อมูลใดๆ Backend ต้อง Query ตรวจสอบเสมอว่า userId ที่ถอดรหัสได้จาก Token เป็นเจ้าของเดียวกับ resourceId (เช่น รหัสการจอง) ที่อ้างอิงอยู่หรือไม่
    - แนะนำให้ใช้ UUID แทน Auto-increment ID เพื่อป้องกันการคาดเดาหมายเลขรายการของบุคคลอื่น

10.3.5.3 Mass Assignment Testing

- OWASP Top 10 (2025)

A01 Broken Access Control (BOLA)

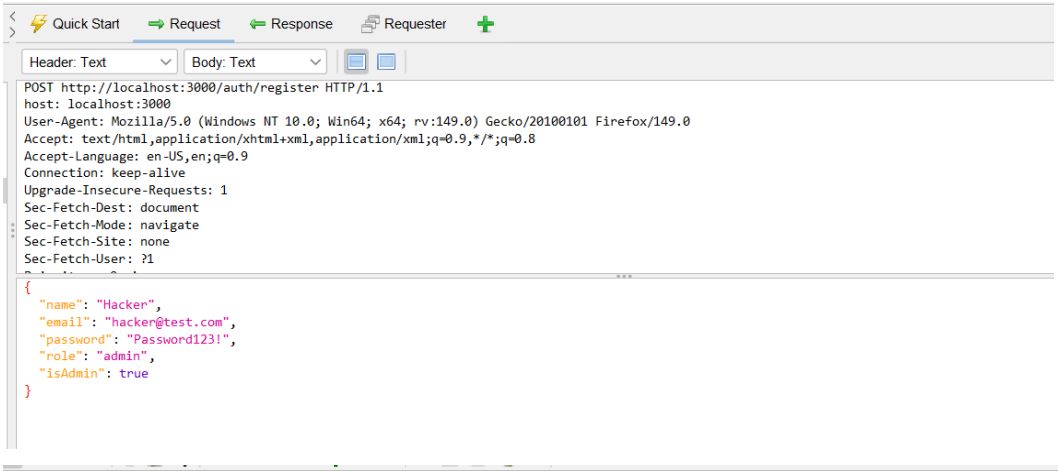
- ระดับความเสี่ยง (CVSS v3.1)

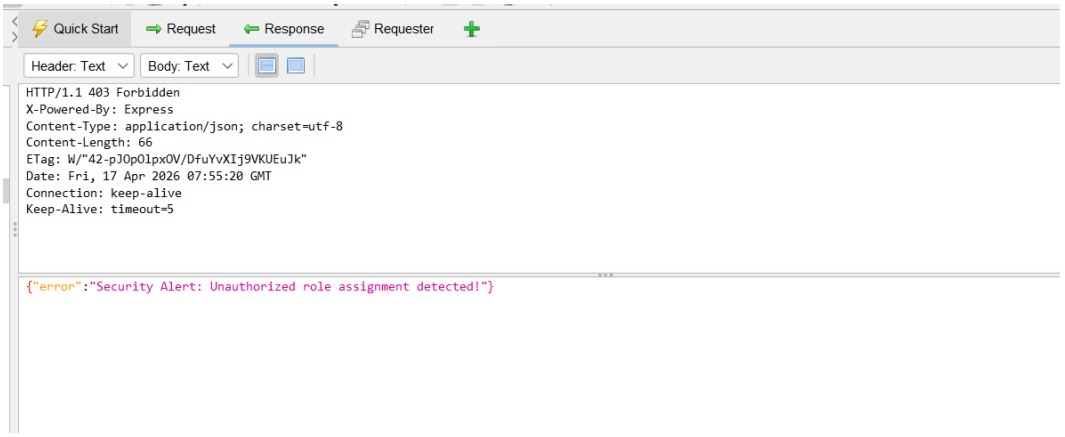
High (8.8)

Metric	Value	Reason
Attack Vector (AV)	Network	แอบเพิ่มฟิลด์ใน JSON ระหว่างลงทะเบียน จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	Low	ใช้การเพิ่ม Key เช่น "role": "admin"
Privileges Required (PR)	None	ไม่จำเป็นต้องมีสิทธิ์หรือบัญชีเข้าใช้งานมาก่อน

Metric	Value	Reason
User Interaction (UI)	None	สามารถยิง api ผ่าน payload ได้โดยตรง
Scope (S)	Unchanged	ขอบเขตการโจมตีจำกัดอยู่แค่บัญชีตนเอง
Confidentiality (C)	High	สามารถเข้าถึงข้อมูลลับของแอดมินได้
Integrity (I)	High	สามารถแก้ไขระบบในฐานะแอดมิน
Availability (A)	High	สามารถปิดระบบหรือลบบัญชีคนอื่นได้

- ขั้นตอนการทดสอบ (Test Steps)
    1. สมัครสมาชิกโดยส่ง "role": "admin" ไปใน Request Body
- ผลการทดสอบที่คาดหวัง (Expected Result)
    - ระบบต้องเพิกเฉยค่า role นั้น หรือปฏิเสธ Request
- ผลการทดสอบ





- แนวทางการแก้ไข
    - ใช้หลักการ Whitelist โดยรับเฉพาะฟิลด์ที่ได้รับอนุญาตเท่านั้น (เช่น email, password) ไม่นำทั้งก้อน Object req.body ไปบันทึกลงฐานข้อมูลโดยตรง
    - กำหนดค่าเริ่มต้น (Default Value) ของ Role ให้เป็นระดับผู้ใช้ทั่วไป (Customer) เสมอที่ฝั่ง Database หรือ Backend

10.3.6 Sensitive Data Protection

- OWASP Top 10 (2025)

A02 Cryptographic Failures

- ระดับความเสี่ยง (CVSS v3.1)

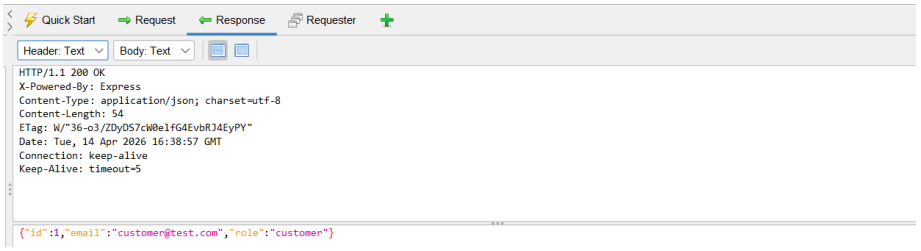
High (7.5)

Metric	Value	Reason
Attack Vector (AV)	Network	ผู้โจมตีสามารถยิง Request จากระยะไกลผ่านอินเทอร์เน็ตได้โดยตรง
Attack Complexity (AC)	Low	เข้าถึงได้ง่ายผ่านเครือข่าย เพียงแค่เรียก API ปกติ (เช่น GET /users) ข้อมูลก็จะหลุดออกมา
Privileges Required (PR)	None	สามารถเรียกดู API ดึงข้อมูลนี้ได้โดยไม่จำเป็นต้องมีบัญชีผู้ใช้งาน หรือไม่ต้องผ่านการล็อกอินเข้าสู่ระบบ (API เปิดสาธารณะ)
User Interaction (UI)	None	ไม่ต้องอาศัย Interact จาก User อื่น
Scope (S)	Unchanged	ผลกระทบจำกัดอยู่แค่ภายในระบบฐานข้อมูลของเว็บไซต์เอง ไม่ได้เปิดช่องให้ผู้โจมตีเจาะทะลุข้ามไประบบอื่นๆ
Confidentiality (C)	High	เข้าถึงข้อมูลที่ระบบพ่นออกมาทาง API เช่น รหัสผ่านหรือข้อมูลส่วนบุคคล
Integrity (I)	None	ไม่ส่งผลให้ Database ถูกแก้ไข
Availability (A)	None	ระบบยังทำงานได้ตามปกติ

- ขั้นตอนการทดสอบ (Test Steps)
    1. ตรวจสอบข้อมูล JSON จาก API Response ของ /users/profile ว่ามีฟิลด์รหัสผ่านหลุดมาหรือไม่
- ผลการทดสอบที่คาดหวัง ข้อมูล Response Body ต้องไม่มีฟิลด์รหัสผ่านหรือข้อมูลที่ละเอียดอ่อน

- ผลการทดสอบ

จากการตรวจสอบ HTTP Response Body ของ Endpoint /users/profile พบว่าระบบส่งกลับเฉพาะข้อมูลที่จำเป็น ได้แก่ id, email และ role โดย ไม่พบฟิลด์รหัสผ่าน หรือข้อมูลส่วนบุคคลที่มีความอ่อนไหวสูงหลุดออกมา



- แนวทางการแก้ไข ใช้ DTO เพื่อคัดกรองข้อมูลก่อนส่งกลับ และเข้ารหัสผ่าน (Hashing) ในฐานข้อมูล

11. Appendix / Reference

Documentation : SW_Testing_Final_Project_Report

Test Case Design : Test_Case_Design

GitHub Repository: https://github.com/isandwish/sw-testing-project

Presentation Slides: https://canva.link/94j33yk7fpw15aq

12. Conclusion

This project presents a structured software testing approach for the Restaurant Table Reservation System (RTS), which is a mock system designed to simulate real-world application behavior and support comprehensive testing practice.

The testing process is organized into four main testing types:

- **API Testing** validates backend functionality based on REST API specifications. It covers authentication, reservation management, table management, and admin operations. Both positive and negative scenarios were tested to ensure correct response handling, proper status codes, and correct enforcement of role-based access control.

- **Automated Testing** using Playwright focuses on end-to-end validation of critical user workflows across Customer, Staff, and Admin roles. The automated scripts ensure that core functions such as login, reservation lifecycle, and system configuration are consistently working and regression-safe.
- **Performance Testing** evaluates system behavior under high load conditions, including up to 1,000 concurrent users. The system was tested for response time, stability over time, and resource usage. Results confirm that the system remains stable under stress and meets defined performance requirements with no critical failures.
- **Security Testing** verifies system protection against key vulnerabilities aligned with OWASP Top 10. This includes authentication and authorization flaws, token security, input validation issues (SQL Injection, XSS), business logic flaws, and sensitive data exposure prevention. The tests confirm that the system correctly handles unauthorized access attempts and input-based attacks with proper validation and error responses.

In addition to execution-based testing, significant emphasis was placed on **Test Case Design**, where requirements were systematically translated into structured test scenarios. Software testing techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, and Negative Testing were applied to ensure complete functional and non-functional coverage. User workflows for Customer, Staff, and Admin roles were also incorporated to reflect real system usage.

Overall, this testing approach demonstrates how multiple levels of testing work together to ensure software quality, including functional correctness, performance stability, security robustness, and system reliability in a realistic development and testing environment.